

2025 ~ 2026	Verifying We Are Human.	Chaoyi Sun, Liwen Zhang, Tian Zhou
2026 ~ 2027	Human verification passed.	Chaoyi Sun, Liwen Zhang, Fengting Deng

Contents

1	Graph	1
1.1	Diameter of a Tree	1
1.2	Centroid of a Tree	1
1.3	Minimum Spanning Tree	1
1.3.1	Prim	1
1.3.2	Kruskal	1
1.4	LCA	1
1.5	Topological Sorting	1
1.6	Shortest Path	2
1.6.1	Dijkstra	2
1.6.2	SPFA	2
1.7	DCC	2
1.8	Tarjan	2
1.9	Bipartite Graph Matchings	3
1.10	Flow	3
1.10.1	Edmonds-Karp	3
1.10.2	Dinic	3
2	Data Structure	5
2.1	Segment Tree	5
2.2	Fenwick Tree	5
2.3	Bitset	5
2.4	Heavy-Light Decomposition	6
2.5	Longest Chain Decomposition	6
2.6	Link Cut Tree	6
2.7	Treap	7
2.8	Cartesian Tree	8
2.9	Sparse Table	8
2.10	Persistent Data Structure	8
2.10.1	Persistent Segment Tree	8
2.10.2	Persistent Treap	8
2.11	Linear-Basis	9
3	Math	10
3.1	Subset Enumeration	10
3.2	Elementary Arithmetic Bucket	10
3.3	Fraction	10
3.4	Combination	10
3.5	The Sieve of Primes & Euler's Totient Function & Möbius Function	10
3.6	Exgcd	10
3.7	CRT	10
3.8	Lagrange Interpolation	11
3.9	Gaussian Elimination	11
3.10	FFT	11
3.11	NTT	11
3.12	FWT	12
4	Computational Geometry	13
5	String	15
5.1	Hash	15
5.2	KMP	15
5.3	Manacher	15
5.4	Trie	15
5.4.1	Trie	15
5.4.2	01-Trie	15
5.5	Aho-Corasick Automaton	15
5.6	SA	16
5.7	SAM	16
6	Other	17
6.1	Fast I/O	17
6.2	Random Number Generator	17
6.3	Execution	17
6.4	Comparison	17
6.5	Python	17

1 Graph

1.1 Diameter of a Tree

```

1 template <typename T>
2 class Diameter
3 {
4     int n, p;
5     vector <T> dis;
6     vector <vector <pair <int, T>>> ve;
7     void dfs(int u, int fa)
8     {
9         for (const auto& [v, w] : ve[u])
10        {
11            if (v == fa) continue;
12            dis[v] = dis[u] + w;
13            if (dis[v] > dis[p]) p = v;
14            dfs(v, u);
15        }
16    }
17 public:
18     Diameter(int n) : n(n), ve(n + 1) {}
19     void add(int u, int v, T w = 1) {ve[u].push_back({v, w}); ve[v].
20         push_back({u, w});}
21     T calc()
22     {
23         dis.assign(n + 1, 0);
24         p = 1; dfs(1, 0);
25         dis[p] = 0; dfs(p, 0);
26         return dis[p];
27     }
28 };

```

1.2 Centroid of a Tree

```

1 class Centroid
2 {
3     int n;
4     vector <int> sz, w, cen;
5     vector <vector <int>> ve;
6     void dfs(int u, int fa)
7     {
8         sz[u] = 1; w[u] = 0;
9         for (auto v : ve[u])
10        {
11            if (v == fa) continue;
12            dfs(v, u);
13            sz[u] += sz[v];
14            w[u] = max(w[u], sz[v]);
15        }
16        w[u] = max(w[u], n - sz[u]);
17        if (w[u] <= n / 2) cen.push_back(u);
18    }
19 public:
20     Centroid(int n) : n(n), ve(n + 1), sz(n + 1), w(n + 1) {}
21     void add(int u, int v) {ve[u].push_back(v); ve[v].push_back(u);}
22     auto calc()
23     {
24         cen.clear();
25         dfs(1, 0);
26         sort(cen.begin(), cen.end());
27         return cen;
28     }
29 };

```

1.3 Minimum Spanning Tree

1.3.1 Prim

```

1 template <typename T>
2 class MST
3 {
4     int n; T ans;
5     vector <int> vis; vector <vector <int>> g; vector <T> dis;
6     public:
7     MST(int n) : n(n), vis(n + 1, 0), g(n + 1, vector <int> (n + 1, INF))
8         , dis(n + 1, INF) {dis[1] = ans = 0; vis[1] = 1;}
9     void add(int u, int v, T w) {g[u][v] = g[v][u] = w;}
10    T calc()
11    {
12        for (int i = 2; i <= n; ++i) dis[i] = g[1][i];
13        for (int i = 1; i < n; ++i)
14        {
15            int k = 0;
16            for (int j = 1; j <= n; ++j)
17                if (!vis[j] && dis[j] < dis[k]) k = j;
18            vis[k] = 1;
19            ans += dis[k];
20        }
21    }
22 };

```

```

19         for (int j = 1; j <= n; ++j)
20             if (!vis[j] && g[k][j] < dis[j]) dis[j] = g[k][j];
21     }
22     return ans;
23 }
24 };

```

1.3.2 Kruskal

```

1 template <typename T>
2 class MST
3 {
4     int n, m, e_cnt, cnt; T ans;
5     struct node {int u, v; T w;};
6     vector <int> fa; vector <node> g;
7     public:
8     MST(int n, int m) : n(n), m(m), fa(n + 1, 0), g(m + 1) {cnt = e_cnt
9         = ans = 0;}
10    void add(int u, int v, int w) {g[++e_cnt].u = u, g[e_cnt].v = v, g[
11        e_cnt].w = w;}
12    int getfa(int u) {return fa[u] == u ? u : fa[u] = getfa(fa[u]);}
13    T calc()
14    {
15        sort(g.begin(), g.end(), [] (auto &x, auto &y) {return x.w < y.w
16            ;});
17        for (int i = 1; i <= n; ++i) fa[i] = i;
18        for (int i = 1; cnt < n && i <= m; ++i)
19        {
20            int dx = getfa(g[i].u), dy = getfa(g[i].v);
21            if (dx == dy) continue;
22            ans += g[i].w; fa[dx] = dy; ++cnt;
23        }
24        return ans;
25    }
26 };

```

1.4 LCA

```

1 class LCA
2 {
3     static constexpr int lg = 20;
4     int n;
5     vector <int> dep;
6     vector <vector <int>> f, ve;
7     public:
8     LCA(int n) : n(n), ve(n + 1), dep(n + 1), f(n + 1, vector <int> (lg +
9         1, 0)) {}
10    void add(int u, int v) {ve[u].push_back(v); ve[v].push_back(u);}
11    void pre(int u, int fa)
12    {
13        f[u][0] = fa; dep[u] = dep[fa] + 1;
14        for (int i = 1; i < lg; ++i) f[u][i] = f[f[u][i-1]][i-1];
15        for (auto v : ve[u])
16            if (v != fa) pre(v, u);
17    }
18    int query(int u, int v)
19    {
20        if (dep[u] < dep[v]) swap(u, v);
21        for (int i = lg; ~i; --i)
22        {
23            if (dep[f[u][i]] >= dep[v]) u = f[u][i];
24            if (u == v) return u;
25        }
26        for (int i = lg; ~i; --i)
27            if (f[u][i] != f[v][i]) u = f[u][i], v = f[v][i];
28        return f[u][0];
29    }
30 };

```

1.5 Topological Sorting

```

1 class Topo
2 {
3     int n;
4     vector <int> deg;
5     vector <vector <int>> ve;
6     public:
7     Topo(int n) : n(n), ve(n + 1), deg(n + 1, 0) {}
8     void add(int u, int v)
9     {
10        ve[u].push_back(v);
11        ++deg[v];
12    }
13    vector <int> calc()
14    {
15        queue <int> q;
16        vector <int> lst;
17    }
18 };

```

```

18     for (int i = 1; i <= n; ++i)
19         if (!deg[i]) q.push(i);
20     while (!q.empty())
21     {
22         int u = q.front(); q.pop();
23         lst.push_back(u);
24         for (auto v : ve[u])
25         {
26             --deg[v];
27             if (!deg[v]) q.push(v);
28         }
29     }
30     return lst;
31 }
32 };

```

1.6 Shortest Path

1.6.1 Dijkstra

```

1 class dijkstra
2 {
3     int n, m, cnt;
4     vector<int> head, to, nxt, vis;
5     vector<ll> dis, val;
6     public:
7     dijkstra(int n, int m) :
8         n(n), m(m), vis(n + 1, 0), head(n + 1, 0), dis(n + 1, 1e18),
9         to(2 * m + 1, 0), nxt(2 * m + 1, 0), val(2 * m + 1, 0) {cnt = 0;}
10
11     void add(int u, int v, int w)
12     {
13         to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
14         to[++cnt] = u; val[cnt] = w; nxt[cnt] = head[v]; head[v] = cnt;
15     }
16     auto calc(int s)
17     {
18         priority_queue<pair<ll, int>> q;
19         for (int i = 1; i <= n; ++i) vis[i] = 0, dis[i] = INF;
20         q.push({0, s});
21         dis[s] = 0;
22         while (!q.empty())
23         {
24             int u = q.top().second; q.pop();
25             if (vis[u]) continue;
26             vis[u] = 1;
27             for (int i = head[u]; i; i = nxt[i])
28             {
29                 int v = to[i];
30                 if (dis[v] > dis[u] + val[i])
31                 {
32                     dis[v] = dis[u] + val[i];
33                     q.push({-dis[v], v});
34                 }
35             }
36         }
37         return dis;
38     };
39 };

```

1.6.2 SPFA

```

1 class SPFA
2 {
3     int n, m, cnt;
4     vector<int> head, to, nxt, val, vis, times;
5     vector<ll> dis;
6     public:
7     SPFA(int n, int m) :
8         n(n), m(m), times(n + 1, 0), vis(n + 1, 0), head(n + 1, 0), dis(
9         n + 1, INF),
10         to(2 * m + 1, 0), nxt(2 * m + 1, 0), val(2 * m + 1, 0) {cnt = 0;}
11
12     void add(int u, int v, int w)
13     {
14         to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
15         to[++cnt] = u; val[cnt] = w; nxt[cnt] = head[v]; head[v] = cnt;
16     }
17     auto calc(int s)
18     {
19         queue<int> q;
20         for (int i = 1; i <= n; ++i) vis[i] = 0, dis[i] = INF;
21         dis[s] = 0, vis[s] = 1; q.push(s);
22         while (!q.empty())
23         {
24             int u = q.front();
25             q.pop(), vis[u] = 0;
26             for (int i = head[u]; i; i = nxt[i])
27             {
28                 int v = to[i];

```

```

27         if (dis[v] > dis[u] + val[i])
28         {
29             dis[v] = dis[u] + val[i];
30             times[v] = times[u] + 1;
31             if (times[v] >= n) return {-1}; //Negative Cycle
32             if (!vis[v]) q.push(v), vis[v] = 1;
33         }
34     }
35     return dis;
36 }
37 };
38 };

```

1.7 DCC

```

1 class DCC
2 {
3     public:
4     struct Edge {int to, nxt;};
5     vector<Edge> G;
6     vector<int> head, dfn, low, col;
7     vector<bool> cut;
8     stack<int> stk;
9     int cnt, cct, n;
10     void init(int nn = 0)
11     {
12         G.assign(2, {0, 0});
13         n = nn; head.assign(n + 1, 0);
14     }
15     void dfs(int u, int f = -1)
16     {
17         dfn[u] = low[u] = ++cnt;
18         int ch = 0;
19         for (int i = head[u]; i; i = G[i].nxt)
20             if (i != f)
21             {
22                 int v = G[i].to;
23                 if (dfn[v] < dfn[u]) stk.push(i >> 1);
24                 if (!dfn[v])
25                 {
26                     dfs(v, i ^ 1); ++ch;
27                     low[u] = min(low[u], low[v]);
28                     if (low[v] >= dfn[u])
29                     {
30                         int I = 0;
31                         ++cct; cut[u] = true;
32                         do
33                         {
34                             assert(!stk.empty());
35                             I = stk.top(); stk.pop();
36                             col[I] = cct;
37                         }
38                         while (I != (i >> 1));
39                     }
40                 }
41                 else if (dfn[v] < low[u]) low[u] = dfn[v];
42             }
43         if (f == -1 && ch == 1) cut[u] = false;
44     }
45     void tarjan()
46     {
47         cnt = cct = 0;
48         col.assign(G.size() >> 1, 0);
49         dfn.assign(n + 1, 0);
50         low.assign(n + 1, 0);
51         cut.assign(n + 1, false);
52         while (!stk.empty()) stk.pop();
53         for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
54     }
55     void insert(int u, int v)
56     {
57         G.push_back({v, head[u]}); head[u] = G.size() - 1;
58         G.push_back({u, head[v]}); head[v] = G.size() - 1;
59     }
60     void insert(vector<vector<int>> &G1)
61     {
62         for (unsigned u = 1; u < G1.size(); ++u)
63             for (int v : G1[u]) insert(u, v);
64     }
65     bool operator [] (const int &x) const {return cut[x];}
66 };

```

1.8 Tarjan

```

1 class Tarjan
2 {
3     int n, m, cnt, times, scc_cnt;
4     vector<int> head, to, nxt, low, scc, dfn;
5     stack<int> s;
6     void tarjan(int u)

```

```

7  {
8      low[u] = dfn[u] = ++times;
9      s.push(u);
10     for (int i = head[u]; i; i = nxt[i])
11     {
12         int v = to[i];
13         if (!dfn[v])
14         {
15             tarjan(v);
16             low[u] = min(low[u], low[v]);
17         }
18         else if (!scc[v]) low[u] = min(low[u], dfn[v]);
19     }
20     if (low[u] == dfn[u])
21     {
22         ++scc_cnt;
23         while (true)
24         {
25             int x = s.top(); s.pop();
26             scc[x] = scc_cnt;
27             if (x == u) break;
28         }
29     }
30 }
31 public:
32 Tarjan(int n, int m) :
33     n(n), m(m), head(n + 2, 0), low(n + 1, 0), dfn(n + 1, 0), scc(n +
34     1, 0),
35     to(2 * m + 2, 0), nxt(2 * m + 2, 0) {cnt = 1; times = scc_cnt =
36     0;};
37 void add(int u, int v) {to[++cnt] = v; nxt[cnt] = head[u]; head[u] =
38     cnt;};
39 auto calc()
40 {
41     for (int i = 1; i <= n; ++i)
42         if (!dfn[i]) tarjan(i);
43     return scc;
44 }
45 };

```

```

14     int u = q.front(); q.pop();
15     for (int i = head[u]; i; i = nxt[i])
16     {
17         int v = to[i];
18         if (!vis[v] && val[i])
19         {
20             pre[v] = u; edge[v] = i; vis[v] = 1;
21             q.push(v);
22             if (v == t) return 1;
23         }
24     }
25 }
26 return 0;
27 }
28 public :
29 EK(int n, int m, int s, int t) :
30     n(n), m(m), s(s), t(t),
31     vis(n + 1, 0), head(n + 1, 0), pre(n + 1, -1), edge(n + 1, -1),
32     to(m + 1, 0), nxt(m + 1, 0), val(m + 1, 0) {cnt = 1;};
33 void add(int u, int v, T w)
34 {
35     to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
36     to[++cnt] = u; val[cnt] = 0; nxt[cnt] = head[v]; head[v] = cnt;
37 }
38 T calc()
39 {
40     T ans = 0;
41     while (bfs())
42     {
43         T mn = INF;
44         for (int i = t; i != s; i = pre[i]) mn = min(mn, val[edge[i
45         ]]);
46         for (int i = t; i != s; i = pre[i]) val[edge[i]] -= mn, val[
47         edge[i] ^ 1] += mn;
48         ans += mn;
49     }
50     return ans;
51 }
52 };

```

1.9 Bipartite Graph Matchings

```

1  class Matching
2  {
3      int l, r; //the number of left/right side points
4      vector<vector<int>> ve;
5      vector<int> vis, op;
6      bool dfs(int u)
7      {
8          for (auto v : ve[u])
9          {
10             if (vis[v]) continue;
11             vis[v] = 1;
12             if (!op[v] || dfs(op[v])) {op[v] = u; return true;};
13         }
14         return false;
15     }
16     public:
17     Matching(int l, int r) : l(l), r(r), vis(r + 1, 0), op(r + 1, 0), ve
18     (l + 1, {})
19     void add(int u, int v) {ve[u].push_back(v);};
20     int calc()
21     {
22         int ans = 0;
23         for (int i = 1; i <= l; ++i)
24         {
25             vis.assign(r + 1, 0);
26             if (dfs(i)) ++ans;
27         }
28         return ans;
29     }
30 };

```

1.10 Flow

1.10.1 Edmonds-Karp

```

1  template <typename T>
2  class EK
3  {
4      int n, m, s, t, cnt;
5      vector<int> head, to, nxt, vis, pre, edge;
6      vector<T> val;
7      bool bfs()
8      {
9          queue<T> q;
10         for (int i = 1; i <= n; ++i) vis[i] = 0, pre[i] = edge[i] = -1;
11         vis[s] = 1; q.push(s);
12         while (!q.empty())
13         {

```

1.10.2 Dinic

```

1  template <typename T>
2  class Dinic
3  {
4      int n, m, s, t, cnt;
5      vector<int> head, to, nxt, cur, dep;
6      vector<T> val;
7      int bfs()
8      {
9          for (int i = 0; i <= n; ++i) dep[i] = 0, cur[i] = head[i];
10         queue<int> q;
11         q.push(s), dep[s] = 1;
12         while (!q.empty())
13         {
14             int u = q.front(); q.pop();
15             for (int i = head[u]; i; i = nxt[i])
16             {
17                 int v = to[i];
18                 if (val[i] && !dep[v]) q.push(v), dep[v] = dep[u] + 1;
19             }
20         }
21         return dep[t];
22     }
23     T dfs(int u, int t, T flow)
24     {
25         if (u == t) return flow;
26         T ans = 0;
27         for (int &i = cur[u]; i && ans < flow; i = nxt[i])
28         {
29             int v = to[i];
30             if (val[i] && dep[v] == dep[u] + 1)
31             {
32                 int x = dfs(v, t, min(val[i], flow - ans));
33                 if (x) val[i] -= x, val[i ^ 1] += x, ans += x;
34             }
35         }
36         if (ans < flow) dep[u] = -1;
37         return ans;
38     }
39     public :
40     Dinic(int n, int m, int s, int t) :
41         n(n), m(m), s(s), t(t),
42         head(n + 1, 0), cur(n + 1, 0), dep(n + 1, 0),
43         to(m + 1, 0), nxt(m + 1, 0), val(m + 1, 0) {cnt = 1;};
44     void add(int u, int v, T w)
45     {
46         to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
47         to[++cnt] = u; val[cnt] = 0; nxt[cnt] = head[v]; head[v] = cnt;
48     }
49     T calc()
50     {
51         T ans = 0;
52         while (bfs())

```

```

53     {
54         T x;
55         while ((x = dfs(s, t, INF))) ans += x;
56     }
57     return ans;
58 }
59 };

1 class Dinic
2 {
3     typedef long long ll;
4     static const int N=1e6+10, M=4e6+10;
5     static const ll INF=4e18;
6     int cgt, head[N], des[M], nxt[M], dep[N], que[N], H, T, h2[N];
7     ll cap[M];
8     inline ll dfs(const int &u, ll in=INF)
9     {
10         if(u==t)return in; ll out=0;
11         for(int &i=h2[u]; i; i=nxt[i])
12         {
13             int v=des[i];
14             if(cap[i]&&dep[v]==dep[u]+1)
15             {
16                 ll tmp=dfs(v, min(in, cap[i]));
17                 out+=tmp; cap[i]-=tmp; cap[i+1]+=tmp;
18                 if(!(in-=tmp))break;
19             }
20         }
21         return out;out:dep[u]=0;
22     }
23     inline bool bfs()
24     {
25         memset(dep+1, 0, n*sizeof(ll));
26         dep[que[H=T=0]]=s=1;
27         while(H<=T)
28         {
29             int u=que[H++];
30             for(int i=head[u]; i; i=nxt[i])if(cap[i])
31             {
32                 int v=des[i];
33                 if(!dep[v])dep[que[++T]=v]=dep[u]+1;
34             }
35         }
36         return dep[t];
37     }
38 public:
39     int n, s, t;
40     inline void Init(const int &sz=0)
41     {
42         memset(head+1, 0, n*sizeof(ll));
43         cgt=1; n=sz;
44     }
45     inline ll Max_Flow(const int &ss, const int &tt)
46     {
47         s=ss; t=tt; ll ans=0;
48         while(bfs())
49         {
50             memcpy(h2+1, head+1, n*sizeof(ll));
51             ans+=dfs(s, INF);
52         }
53         return ans;
54     }
55     inline bool ins(const int &x, const int &y, const ll &w)
56     {
57         if(x>n||y>n||x<1||y<1)return false;
58         nxt[++cgt]=head[x]; des[head[x]=cgt]=y; cap[cgt]=w;
59         nxt[++cgt]=head[y]; des[head[y]=cgt]=x; cap[cgt]=0;
60         return true;
61     }
62     inline int newNode(){return ++n;}
63 }a;

64
65 namespace MCMF
66 {
67     typedef long long Val_Type; typedef long long Cap_Type;
68     const Val_Type INF_v=4e18; const Cap_Type INF_c=4e18;
69     const int N=1e6+10, M=2e6+10; int head[N], des[M], nxt[M], cgt=1;
70     Cap_Type cap[M]; Val_Type val[M];
71     template<typename T> inline T Abs(T x){return x<0?-x:x;}
72     inline void ins(const int &x, const int &y, const int &c, const
73         Val_Type &w)
74     {
75         nxt[++cgt]=head[x]; des[head[x]=cgt]=y; cap[cgt]=c; val[cgt]=w;
76         nxt[++cgt]=head[y]; des[head[y]=cgt]=x; cap[cgt]=0; val[cgt]=-w;
77     }
78     int que[20000010], H, T; Val_Type dis[N], mnc; int s, t, S; bool inq
79     [N];
80     inline bool spfa()
81     {
82         for(int i=0; i<=S+2; ++i)dis[i]=INF_v; dis[que[H=T=0]]=s=0;
83         while(H<=T)
84         {
85             int u=que[H++]; inq[u]=false; for(int i=head[u]; i; i=nxt[i]

```

```

84         {int v=des[i]; if(cap[i]&&dis[v]>dis[u]+val[i]){dis[v]=
85             dis[u]+val[i]; if(!inq[v])inq[que[++T]=v]=true;}}
86         }
87         return dis[t]<INF_v;
88     }
89     inline Cap_Type dfs(const int &u=s, Cap_Type in=INF_c)
90     {
91         if(u==t)return in; Cap_Type out=0; inq[u]=true;
92         for(int i(head[u]); i; i=nxt[i])
93         {
94             int v=des[i]; if(cap[i]&&!inq[v]&&dis[v]==(dis[u]+val[i]))
95             {
96                 Cap_Type tmp=dfs(v, cap[i]<in?cap[i]:in); mnc+=val[i]*
97                 tmp;
98                 cap[i]-=tmp; cap[i+1]+=tmp; out+=tmp; if(!(in-=tmp))
99                 break;
100             }
101             if(!out)dis[u]=4e18; inq[u]=false; return out;
102         }
103     }
104     inline Cap_Type dinic(){Cap_Type res=0; while(spfa())res+=dfs();
105     return res;}
106     inline void Clear(){memset(head, 0, (S+2)*sizeof(head[0])); cgt=1; S
107     =0; mnc=0;}
108 }
109 using MCMF::ins; using MCMF::s; using MCMF::t; using MCMF::S; using MCMF:
110 :dinic; using MCMF::mnc;

```

2 Data Structure

2.1 Segment Tree

```

1 namespace Seg
2 {
3     struct SEG(int l, r, sum, tag, vl, vr; t[N]; int Rt, cnt;
4     inline void update(int k){t[k].sum=t[t[k].l].sum+t[t[k].r].sum;}
5     inline void pushdown(int k)
6     {
7         if(t[k].l)t[t[k].l].sum+=t[k].tag*(t[t[k].l].vr-t[t[k].l].vl+1),
8         t[t[k].l].tag+=t[k].tag;
9         if(t[k].r)t[t[k].r].sum+=t[k].tag*(t[t[k].r].vr-t[t[k].r].vl+1),
10        t[t[k].r].tag+=t[k].tag; t[k].tag=0;
11    }
12    void add(int cur, int ql, int qr, int val, int l, int r)
13    {
14        int l=t[cur].vl, r=t[cur].vr, mid=(l+r)>>1;
15        if(ql<=l&&r<=qr){t[cur].sum+=val*(r-l+1); t[cur].tag+=val; return;}
16
17        pushdown(cur); if(ql<=mid)add(t[cur].l, ql, qr, val, l, mid);
18        if(qr>mid)add(t[cur].r, ql, qr, val, mid+1, r); update(cur);
19    }
20    int query(int cur, int ql, int qr, int l, int r)
21    {
22        int res=0, l=t[cur].vl, r=t[cur].vr, mid=(l+r)>>1;
23        pushdown(cur); if(ql<=l&&r<=qr){return t[cur].sum;}
24        if(ql<=mid)res+=query(t[cur].l, ql, qr, l, mid);
25        if(qr>mid)res+=query(t[cur].r, ql, qr, mid+1, r); update(cur); return
26        res;
27    }
28    void build(int l, int r, int &cur)
29    {
30        t[cur]=++cnt; t[cur].vl=l; t[cur].vr=r; if(l==r){t[cur].sum=a[l]; return;}
31        int mid=(l+r)>>1; if(l<=mid)build(l, mid, t[cur].l);
32        if(r>mid)build(mid+1, r, t[cur].r); update(cur);
33    }
34    int find(int cur, int ql, int qr, int x, int l, int r)
35    {
36        int l=t[cur].vl, r=t[cur].vr, mid=(l+r)>>1;
37        if(ql<=l&&r<=qr)
38        {
39            if(t[cur].sum<x)return -1;
40            if(l==r)return l;
41            pushdown(cur);
42            if(t[t[cur].l].sum>=x)return find(t[cur].l, ql, qr, x, l, mid);
43            return find(t[cur].r, ql, qr, x-t[t[cur].l].sum, mid+1, r);
44        }
45        pushdown(cur);
46        if(qr<=mid)return find(t[cur].l, ql, qr, x, l, mid);
47        if(ql>mid)return find(t[cur].r, ql, qr, x, mid+1, r);
48        int res=find(t[cur].l, ql, qr, x, l, mid);
49        if(res!=-1)return res;
50        int sum=query(t[cur].l, ql, mid, l, mid);
51        return find(t[cur].r, ql, qr, x-sum, mid+1, r);
52    }
53    inline int merge(int x, int y, int l, int r)
54    {
55        if(!x||!y)return x|y;
56        if(l==r){t[x].sm+=t[y].sm; return x;}
57        int mid=(l+r)>>1;
58        t[x].l=merge(t[x].l, t[y].l, l, mid);
59        t[x].r=merge(t[x].r, t[y].r, mid+1, r);
60        update(x); return x;
61    }
62    //Persistent
63    inline void merge(int &x, int y, int l=1, int r=N-10)
64    {
65        if(!x||!y){x=y; return;}
66        int nx=++cnt; t[nx]=t[x]; x=nx;
67        if(l==r){t[x].mx+=t[y].mx; t[x].res=t[x].mx?1:0; return;}
68        int mid=(l+r)>>1;
69        merge(t[x].l, t[y].l, l, mid); merge(t[x].r, t[y].r, mid+1, r);
70        update(x);
71    }
72 }

```

2.2 Fenwick Tree

```

1 inline void add(int *t, int p, int v){while(p<=n)t[p]+=v, p+=(p&p);}
2 inline int qry(int *t, int p){int res=0; while(p)res+=t[p], p^=(p&p);
3     return res;}
4 inline int qry(int *t, int k){return n+1 if all satisfy
5 {
6     int p=0, tot=0;
7     for(int i=20; i; --i){if((p|1<<i)<=n&&tot+t[p|(1<<i)]<k)tot+=t[p|(1<<
8     i)];
9     return p+1;
10 }

```

2.3 Bitset

```

1 class Bitset{
2     int n, m;
3     vector<uint64_t> a;
4
5     static inline uint64_t rev(uint64_t x){
6         x=((x&0x5555555555555555ULL)<<1)|((x>>1)&0x5555555555555555ULL);
7         x=((x&0x3333333333333333ULL)<<2)|((x>>2)&0x3333333333333333ULL);
8         x=((x&0xf0f0f0f0f0f0f0f0ULL)<<4)|((x>>4)&0xf0f0f0f0f0f0f0f0ULL);
9         x=((x&0x00ff00ff00ff00ff00ULL)<<8)|((x>>8)&0x00ff00ff00ff00ff00ULL);
10        x=((x&0x0000ffff0000ffffULL)<<16)|((x>>16)&0x0000ffff0000ffffULL);
11    };
12    return (x<<32)|(x>>32);
13 }
14
15 inline void fix(){if(n&63)a.back()&=((1ULL<<(n&63))-1);}
16 //set i>=n to zero
17
18 public:
19     class Ref{
20         uint64_t&x, m;
21     public:
22         Ref(uint64_t&x, uint64_t m):x(x), m(m){}
23         Ref&operator=(bool v){if(v)x|=m; else x&=~m; return *this;}
24         Ref&operator=(const Ref&v){return *this=Ref(v);}
25         operator bool(){return x&m;}
26         Ref&flip(){x^=m; return *this;}
27     };
28
29     Bitset(int n=0):n(n), m((n+63)>>6), a(m){}
30
31     Ref operator[](int p){return Ref(a[p>>6], 1ULL<<(p&63));}
32     bool operator[](int p)const{return (a[p>>6]>>(p&63))&1;}
33     bool test(int p)const{return (*this)[p];}
34
35     Bitset&set(){
36         fill(a.begin(), a.end(), ~0ULL);
37         fix(); return *this;
38     }
39     Bitset&set(int p, bool v=1){(*this)[p]=v; return *this;}
40     Bitset&reset(){fill(a.begin(), a.end(), 0); return *this;}
41     Bitset&reset(int p){(*this)[p]=0; return *this;}
42
43     Bitset&flip(){
44         for(auto&x:a)x=~x;
45         fix(); return *this;
46     }
47     Bitset&flip(int p){a[p>>6]^=1ULL<<(p&63); return *this;}
48
49     bool any()const{
50         for(auto x:a)if(x)return 1;
51         return 0;
52     }
53     bool none()const{return !any();}
54
55     int count()const{
56         int s=0;
57         for(auto x:a)s+=__builtin_popcountll(x);
58         return s;
59     }
60
61     int size()const{return n;}
62
63     Bitset&operator&=(const Bitset&b){
64         for(int i=0; i<m; i++)a[i]&=b.a[i];
65         return *this;
66     }
67     Bitset&operator|=(const Bitset&b){
68         for(int i=0; i<m; i++)a[i]|=b.a[i];
69         return *this;
70     }
71     Bitset&operator^=(const Bitset&b){
72         for(int i=0; i<m; i++)a[i]^=b.a[i];
73         return *this;
74     }
75
76     Bitset&operator<=(int k){
77         if(k>=n)return reset();
78         int q=k>>6, r=k&63;
79         if(q){
80             for(int i=m-1; i>=q; i--)a[i]=a[i-q];
81             for(int i=0; i<q; i++)a[i]=0;
82         }
83         if(r){
84             for(int i=m-1; i-->0)a[i]=(a[i]<<r)|(a[i-1]>>(64-r));
85             a[0]<<=r;
86         }
87         fix(); return *this;
88     }
89
90     Bitset&operator>=(int k){
91         if(k>=n)return reset();
92         int q=k>>6, r=k&63;
93         if(q){

```

```

93     for(int i=0;i<q<m;i++)a[i]=a[i+q];
94     for(int i=m-q;i<m;i++)a[i]=0;
95 }
96 if(r){
97     for(int i=0;i+1<m;i++)a[i]=(a[i]>>r)|(a[i+1]<<(64-r));
98     a[m-1]>>=r;
99 }
100 return *this;
101 }
102
103 Bitset&reverse(){
104     for(int i=0,j=m-1;i<=j;i++,j--){
105         uint64_t x=rev(a[i]);
106         a[i]=rev(a[j]);
107         a[j]=x;
108     }
109     if(n&63){
110         int r=n&63,s=64-r;
111         for(int i=0;i+1<m;i++)a[i]=(a[i]>>s)|(a[i+1]<<r);
112         a[m-1]>>=s;
113     }
114     fix();return *this;
115 }
116 Bitset&operator+=(const Bitset&b){
117     uint64_t c=0;
118     for(int i=0;i<m;i++){
119         uint64_t x=a[i],y=b.a[i];
120         a[i]=x+y+c;
121         c=(a[i]<x)|| (c&&a[i]==x);
122     }
123     fix();
124     return *this;
125 }
126
127 Bitset&operator-=(const Bitset&b){
128     uint64_t c=0;
129     for(int i=0;i<m;i++){
130         uint64_t x=a[i],y=b.a[i];
131         a[i]=x-y-c;
132         c=(x<y)|| (c&&x==y);
133     }
134     fix();
135     return *this;
136 }
137 friend Bitset operator|(Bitset x,const Bitset&y){return x&=y;}
138 friend Bitset operator&(Bitset x,const Bitset&y){return x|=y;}
139 friend Bitset operator^(Bitset x,const Bitset&y){return x^=y;}
140 friend Bitset operator<<(Bitset x,int k){return x<<=k;}
141 friend Bitset operator>>(Bitset x,int k){return x>>=k;}
142 friend Bitset operator~(Bitset x){return x.flip();}
143 };

```

2.4 Heavy-Light Decomposition

```

1 class HLD
2 {
3     public:
4         vector<vector<int>> > G;
5         int n, rt, cdt;
6         HLD() : n(0), rt(1), cdt(0) {}
7         vector<int> top, dep, fa, mxs, siz, dfn;
8         void dfs1(int u, int f)
9         {
10             siz[u] = 1;
11             fa[u] = f;
12             dep[u] = dep[f] + 1;
13             int mx = 0;
14             for(int v : G[u]) if(v != f)
15             {
16                 dfs1(v, u);
17                 siz[u] += siz[v];
18                 if(siz[v] > mx)
19                 {
20                     mx = siz[v];
21                     mxs[u] = v;
22                 }
23             }
24         }
25         void dfs2(int u, int f, int tp)
26         {
27             if(mxs[u]) dfs2(mxs[u], u, tp);
28             top[u] = tp;
29             dfn[u] = ++cdt;
30             // subtree : [dfn[u], dfn[u] + siz[u] - 1]
31             // chain : [dfn[top[u]], dfn[u]]
32             for(int v : G[u]) if(v != f && v != mxs[u])
33             {
34                 dfs2(v, u, v);
35             }
36         }
37
38         HLD(vector<vector<int>> &T, int RT = 1)
39         {

```

```

40             G = T;
41             rt = RT;
42             n = int(T.size()) - 1;
43             top.assign(n+1, 0);
44             dep.assign(n+1, 0);
45             fa.assign(n+1, 0);
46             mxs.assign(n+1, 0);
47             siz.assign(n+1, 0);
48             dfn.assign(n+1, 0);
49             dfs1(rt, 0);
50             cdt = 0;
51             dfs2(rt, 0, rt);
52         }
53         int LCA(int u, int v)
54         {
55             while(top[u] != top[v])
56             {
57                 dep[top[u]] > dep[top[v]] ? u = fa[top[u]] : v = fa[top[
58                 v]];
59             }
60             return dep[u] > dep[v] ? v : u;
61         }
62 };

```

2.5 Longest Chain Decomposition

```

1 //Total Length of Chains is O(n)
2 //Each merge will eliminate 1 node of the shorter chains
3 const int N = 1e6 + 10;
4 long long F[N]; //f[rt][i]: the sum of nodes at distance exactly i from
5                 rt
6 //e.g. the sum of the nodes with distance = i
7 int mxs[N], dep[N], mxd[N];
8 long long a[N];
9 vector<int> G[N];
10 int n, rt;
11 void dfs(int u = 1, int fa = 0)
12 {
13     dep[u] = dep[fa] + 1;
14     int mx = -1;
15     mxd[u] = dep[u];
16     for(int v : G[u]) if(v != fa)
17     {
18         dfs(v, u);
19         mxd[u] = max(mxd[u], mxd[v]);
20         if(mxd[v] > mx)
21         {
22             mx = mxd[v];
23             mxs[u] = v;
24         }
25     }
26 }
27 void DP(int u = 1, int fa = 0, long long *f = F)
28 {
29     f[0] += a[u];
30     long long *tmp = f + (mxd[u] - dep[u] + 1);
31     for(int v : G[u]) if(v != fa)
32     {
33         if(v != mxs[u])
34         {
35             long long *fv = tmp;
36             DP(v, u, fv);
37             for(int i = 0; i <= mxd[v] - dep[v]; ++i)
38             {
39                 f[i + 1] += fv[i];
40                 fv[i] = 0;
41             }
42         }
43         else
44         {
45             DP(v, u, f + 1);
46         }
47     }
48 }

```

2.6 Link Cut Tree

```

1 const int N = 1e5 + 10;
2 struct Splay {
3     int s[2] = {0, 0};
4     int fa = 0;
5     int sum = 0;
6     int vl = 0;
7     bool tg = false;
8 } t[N];
9 int n;
10 // Check whether x is not the root of its auxiliary tree.
11 // isSon(x) == false : is Root
12 inline bool isSon(const int &x) {return t[t[x].fa].s[0] == x || t[t[x].
13                 fa].s[1] == x;}

```



```

13 inline void rev(const int &x) {if (!x) return; swap(t[x].s[0], t[x].s[1]);
    t[x].tg ^= 1;}
14 inline void update(const int &x) {t[x].sum = t[t[x].s[0]].sum ^ t[t[x].s
    [1]].sum ^ t[x].v1;}
15 inline void pushdown(const int &x) {if (!t[x].tg) return; t[x].tg = false;
    rev(t[x].s[0]); rev(t[x].s[1]);}

16
17 // Rotate x in its auxiliary tree.
18 inline void rotate(const int &x) {
19     int y = t[x].fa;
20     int z = t[y].fa;
21     bool k = (t[y].s[1] == x);
22     if (isSon(y)) t[z].s[t[z].s[1] == y] = x;
23     int w = t[x].s[k ^ 1];
24     t[x].s[k ^ 1] = y;
25     t[y].s[k] = w;
26     if (w) t[w].fa = y;
27     t[x].fa = z;
28     t[y].fa = x;
29     update(y);
30     update(x);
31 }
32 int sk[N], tp;
33 // Splay x and push all lazy tags on the path to x.
34 inline void splay(const int x) {
35     tp = 1;
36     sk[tp] = x;
37     while (isSon(sk[tp])) {
38         ++tp;
39         sk[tp] = t[sk[tp - 1]].fa;
40     }
41     while (tp) pushdown(sk[tp--]);
42     while (isSon(x)) {
43         int y = t[x].fa;
44         if (isSon(y)) {
45             bool zigZig = (t[y].s[0] == x) == (t[t[y].fa].s[0] == y);
46             rotate(zigZig ? y : x);
47         }
48         rotate(x);
49     }
50     update(x);
51 }
52
53 // Expose the preferred path from x to the origin-tree root.
54 inline void access(int x) {
55     for (int y = 0; x; x = t[y = x].fa) {
56         splay(x);
57         t[x].s[1] = y;
58         update(x);
59     }
60 }
61
62 // Make x the root of its original tree.
63 inline void makeRt(const int &x) {
64     access(x);
65     splay(x);
66     rev(x);
67 }
68
69 // Find the represented-tree root containing x.
70 inline int fndRt(int x) {
71     access(x);
72     splay(x);
73     while (t[x].s[0]) {
74         pushdown(x);
75         x = t[x].s[0];
76     }
77     splay(x);
78     return x;
79 }
80
81 // Expose the path from x to y in y's auxiliary tree. (Exactly x->y,
    Rooted at y)
82 inline void split(const int &x, const int &y) {
83     makeRt(x);
84     access(y);
85     splay(y);
86 }
87
88 // Link the trees containing x and y.
89 inline void lnk(const int &x, const int &y) {
90     makeRt(x);
91     if (fndRt(y) != x) t[x].fa = y;
92 }
93
94 // Cut the edge x-y if it exists.
95 inline void cut(const int &x, const int &y) {
96     makeRt(x);
97     if (fndRt(y) == x && t[y].fa == x && !t[y].s[0]) {
98         t[y].fa = t[x].s[1] = 0;
99         update(x);
100     }
101 }
102
103 // Modify the value stored at x.
104 inline void modify(const int &x, const int &value) {

```

```

105     access(x);
106     splay(x);
107     t[x].v1 = value;
108     update(x);
109 }

```

2.7 Treap

```

1 namespace VAL
2 {
3     inline void update(int k){t[k].siz=t[t[k].l].siz+t[t[k].r].siz+1;}
4     inline int new_node(int k){t[++cnt].val=k;t[cnt].siz=1;t[cnt].key=
        rand();return cnt;}
5     inline void split(int k,int cur,int &x,int &y)
6     {
7         if(!cur){x=y=0;return;}pushdown(cur);
8         else if(t[cur].val<=k)x=cur,split(k,t[cur].r,t[cur].r,y);
9         else y=cur,split(k,t[cur].l,x,t[cur].l);update(cur);
10    }
11    inline int merge(int L,int R)
12    {
13        if(!L||!R)return L+R;
14        pushdown(L);pushdown(R);
15        if(t[L].key<t[R].key)
16        {
17            t[L].r=merge(t[L].r,R);
18            update(L);return L;
19        }
20        else
21        {
22            t[R].l=merge(L,t[R].l);
23            update(R);return R;
24        }
25    }
26 }
27 namespace SIZ
28 {
29     inline void split(int k,int cur,int &x,int &y)
30     {
31         if(!cur){x=y=0;return;}pushdown(cur);
32         if(k<=t[t[cur].l].siz){split(k,t[cur].l,x,t[y=cur].l);update(cur);
            return;}
33         else {split(k-t[t[cur].l].siz-1,t[cur].r,t[x=cur].r,y);update(
            cur);return;}
34    }
35    inline int merge(int L,int R)
36    {
37        if(!L||!R)return L+R;pushdown(L);pushdown(R);
38        if(t[L].key<t[R].key){t[L].r=merge(t[L].r,R);update(L);return L;}
39        else{t[R].l=merge(L,t[R].l);update(R);return R;}
40    }
41    inline int rnk(int k){int res=t[t[k].l].siz;while(t[k].f){res+=(k==t
        [t[t[k].f].l].siz+1:0;k=t[k].f);}return res+1;}
42 }
43 namespace MERGE//merge same values into one node
44 {
45     struct Node
46     {
47         int l,r,f,ct,sz,v1;
48         Node(){l=r=v1=sz;}
49         Node(int v,int c){l=r=f;ct=sz=c;v1=v;}
50     }t[N];
51     int n,stk[N],top,cnt,T1,T2,T3,T4,T5,Rt;
52     inline int node(int v,int c)
53     {
54         int u=top?stk[top--]:++cnt;
55         t[u]=Node(v,c);return u;
56     }
57     inline int copy(int x)
58     {
59         int u=top?stk[top--]:++cnt;
60         t[u]=t[x];return u;
61     }
62     inline void update(int x)
63     {
64         pushdown(x);
65         t[x].sz=t[t[x].l].sz+t[t[x].r].sz+t[x].ct;
66         if(t[x].l)t[t[x].l].f=x;
67         if(t[x].r)t[t[x].r].f=x;
68         t[x].f=0;
69     }
70     inline void split(int cur,int k,int &x,int &y)
71     {
72         if(!cur){x=y=0;return;}
73         pushdown(cur);
74         if(t[t[cur].l].sz>=k)
75         {
76             y=cur;
77             split(t[t[cur].l].l,k,x,t[y].l);
78             update(y);
79         }
80     }

```

```

81     else if(t[t[cur].l].sz+t[cur].ct>=k)
82     {
83         k-=t[t[cur].l].sz;
84         if(k==t[cur].ct)
85         {
86             y=t[cur].r;
87         }
88         else
89         {
90             y=copy(cur);
91             t[y].ct-=k;
92             t[y].l=0;
93         }
94         x=cur; t[x].ct=k;
95         t[x].r=0;
96         update(x);
97         update(y);
98     }
99     else
100    {
101        x=cur;
102        split(t[cur].r, k-t[t[cur].l].sz-t[cur].ct, t[x].r, y);
103        update(x);
104    }
105 }
106 inline int merge(int x, int y)
107 {
108     if(!x||!y) return x||y;
109     pushdown(x); pushdown(y);
110     if(int(rnd())%(t[x].sz+t[y].sz)<t[x].sz)
111     {
112         t[x].r=merge(t[x].r, y);
113         update(x);
114         return x;
115     }
116     else
117     {
118         t[y].l=merge(x, t[y].l);
119         update(y);
120         return y;
121     }
122 }
123 }

```

2.8 Cartesian Tree

```

1 //heap-order on values and BST-order on keys
2 stack<int> s;
3 vector<int> L(n+1, 0), R(n+1, 0);
4 for (int i = 1; i <= n; ++i)
5 {
6     int la = -1;
7     while (!s.empty() && a[s.top()] > a[i]) la = s.top(), s.pop();
8     if (!s.empty()) R[s.top()] = i;
9     if (la != -1) L[i] = la;
10    s.push(i);
11 }

```

2.9 Sparse Table

```

1 vector<int> lg(n+1, -1);
2 vector<vector<int>> f(20, vector<int>(n+1, INF));
3 for (int i = 1; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
4 for (int i = 1; i <= n; ++i) f[0][i] = a[i];
5 for (int j = 1; j < 20; ++j)
6     for (int i = 1; i + (1 << (j - 1)) <= n; ++i) f[j][i] = min(f[j-1][i], f[j-1][i + (1 << (j - 1))]);
7 auto query = [&](int l, int r) -> int {return min(f[lg[r-l+1]][l], f[lg[r-l+1]][r - (1 << lg[r-l+1]) + 1]);};

```

2.10 Persistent Data Structure

2.10.1 Persistent Segment Tree

```

1 struct Seg{int l, r, val; }t[N*64]; int cnt, a[N], Rt[N], dep[N], Mx[N];
2 inline void build(int &x, int l, int r)
3 {
4     int mid=(l+r)>>1; x=++cnt; if(l==r){t[x].val=a[l]; return;}
5     build(t[x].l, l, mid); build(t[x].r, mid+1, r); return;
6 }
7 inline void Fix(int cur, int &x, int loc, int val, int l, int r)
8 {
9     t[x=++cnt]=t[cur]; int mid=(l+r)>>1;
10    if(l==r){t[x].val=val; dep[x]=dep[cur]; return;}
11    if(loc<=mid)Fix(t[cur].l, t[x].l, loc, val, l, mid);
12    else Fix(t[cur].r, t[x].r, loc, val, mid+1, r);
13 }

```

```

14 inline int Query(int cur, int loc, int l, int r)
15 {
16     int mid=(l+r)>>1; if(l==r) return t[cur].val;
17     if(loc<=mid) return Query(t[cur].l, loc, l, mid);
18     return Query(t[cur].r, loc, mid+1, r);
19 }

```

2.10.2 Persistent Treap

```

1 namespace PFHQ{//Interval Copy
2 {
3     #include<bits/stdc++.h>
4     #define rep(a,b,c) for(register int c=(a);c<=(b);++c)
5     using namespace std;
6     const int Mod=1e9+7, INF=0x3f3f3f3f, N=4e6+5;
7     inline int read()
8     {
9         int res=0; char ch=getchar(); while(ch<'0' || ch>'9') ch=getchar();
10        while(ch<='9' && ch>='0') res=res*10+(ch-'0'), ch=getchar(); return res;
11    }
12    inline int add(int &x, int y){x=((x+y>=Mod)?(x+y-Mod):(x+y));}
13    struct FHQ{int l, r, siz, val, sum, tag[3]; }t[N];
14    int a[N], cnt, Root, top, n, m, T1, T2, T3, T4, T5;
15    inline int Node(int val)
16    {
17        t[++cnt].siz=1; t[cnt].sum=t[cnt].val=val; t[cnt].tag[0]=INF;
18        t[cnt].l=t[cnt].r=t[cnt].tag[1]=t[cnt].tag[2]=0; return cnt;
19    }
20    inline int copy(int x){t[++cnt]=t[x]; return cnt;}
21    inline void update(int x)
22    {
23        t[x].siz=t[t[x].l].siz+t[t[x].r].siz+1;
24        t[x].sum=((t[t[x].l].sum+t[t[x].r].sum)%Mod+t[x].val)%Mod;
25    }
26    inline void push2(int x){swap(t[x].l, t[x].r); t[x].tag[2]^=1;}
27    inline void push1(int x, int k){add(t[x].val, k); add(t[x].tag[1], k); t[x].sum=
28    (t[x].sum+1ll*t[x].siz*k%Mod)%Mod;}
29    inline void push0(int x, int k)
30    {t[x].tag[1]=0; t[x].tag[0]=t[x].val=k; t[x].sum=1ll*t[x].siz*k%Mod;}
31    inline void pushdown(int cur)
32    {
33        if(t[cur].tag[0]!=INF || t[cur].tag[1] || t[cur].tag[2])
34        {
35            if(t[cur].l) t[cur].l = copy(t[cur].l);
36            if(t[cur].r) t[cur].r = copy(t[cur].r);
37        }
38        if(t[cur].tag[0]!=INF)
39        {
40            if(t[cur].l) push0(t[cur].l, t[cur].tag[0]);
41            if(t[cur].r) push0(t[cur].r, t[cur].tag[0]);
42            t[cur].tag[0] = INF;
43        }
44        if(t[cur].tag[1])
45        {
46            if(t[cur].l) push1(t[cur].l, t[cur].tag[1]);
47            if(t[cur].r) push1(t[cur].r, t[cur].tag[1]);
48            t[cur].tag[1] = 0;
49        }
50        if(t[cur].tag[2]){if(t[cur].l)push2(t[cur].l); if(t[cur].r)push2(t[cur].r); t[cur].tag[2]=0;}
51    }
52    inline int merge(int x, int y)
53    {
54        if(!x||!y) return x||y; pushdown(x); pushdown(y);
55        if(rand()%(t[x].siz+t[y].siz)<t[x].siz){x=copy(x); t[x].r=merge(t[x].r, y); update(x); return x;}
56        y=copy(y); t[y].l=merge(x, t[y].l); update(y); return y;
57    }
58    inline void split(int cur, int k, int &x, int &y)
59    {
60        if(!cur){x=y=0; return;} pushdown(cur);
61        if(t[t[cur].l].siz>=k)y=copy(cur), split(t[y].l, k, x, t[y].l), update(y);
62        else x=copy(cur), split(t[x].r, k-t[t[x].l].siz-1, t[x].r, y), update(x);
63    }
64    inline void build(int l, int r, int &cur)
65    {
66        int mid=(l+r)>>1; cur=Node(a[mid]);
67        if(l<mid) build(l, mid-1, t[cur].l);
68        if(r>mid) build(mid+1, r, t[cur].r); update(cur);
69    }
70    inline void change(int l, int r, int k)
71    {
72        split(Root, l-1, T1, T2); split(T2, r-1+1, T2, T3);
73        T2=copy(T2); push0(T2, k); Root=merge(merge(T1, T2), T3);
74    }
75    inline void modify(int l, int r, int k)
76    {
77        split(Root, l-1, T1, T2); split(T2, r-1+1, T2, T3);
78        T2=copy(T2); push1(T2, k); Root=merge(merge(T1, T2), T3);
79    }
80 }

```

```

80 inline void Copy(int l1,int r1,int l2,int r2)//swap
81 {
82     if(l1>l2)swap(l1,l2),swap(r1,r2);
83     split(Root,l1-1,T1,T2);split(T2,r1-l1+1,T2,T3);
84     split(T3,l2-r1+1,T3,T4);split(T4,r2-l2+1,T4,T5);
85     Root=merge(merge(merge(merge(T1,T4),T3),T2),T5);
86 }
87 inline void paste(int l1,int r1,int l2,int r2)//copy
88 {
89     bool flag = 0;if(l1>l2)swap(l1,l2),swap(r1,r2),flag=1;
90     split(Root,l1-1,T1,T2);split(T2,r1-l1+1,T2,T3);
91     split(T3,l2-r1+1,T3,T4);split(T4,r2-l2+1,T4,T5);
92     if(flag) T2=copy(T4);else T4=copy(T2);
93     Root=merge(merge(merge(merge(T1,T2),T3),T4),T5);
94 }
95 inline int query(int l,int r)
96 {
97     split(Root,l-1,T1,T2);split(T2,r-1+1,T2,T3);
98     int res=t[T2].sum;Root=merge(merge(T1,T2),T3);return res;
99 }
100 inline void dfs(int cur)
101 {
102     pushdown(cur);if(t[cur].l)dfs(t[cur].l);
103     a[++top]=t[cur].val;if(t[cur].r)dfs(t[cur].r);
104 }
105 inline void reverse(int l,int r)
106 {
107     split(Root,l-1,T1,T2);split(T2,r-1+1,T2,T3);
108     T2=copy(T2);push2(T2);Root=merge(merge(T1,T2),T3);
109 }
110 int main()
111 {
112     srand(time(0));
113     n=read();m=read();rep(1,n,i)a[i]=read();build(1,n,Root);rep(1,m,
114 i)
115 {
116     int opt=read(),l1=read(),r1=read(),l2,r2;
117     switch(opt)
118     {
119         case 1: printf("%d\n",query(l1,r1));break;
120         case 2: change(l1,r1,read());break;
121         case 3: modify(l1,r1,read());break;
122         case 4: l2=read();r2=read();paste(l1,r1,l2,r2);break;
123         case 5: l2=read();r2=read();Copy(l1,r1,l2,r2);break;//
124 swap
125         case 6: reverse(l1,r1);break;
126     }
127     if(cnt>N-1000000)dfs(Root),top=Root=cnt=0,build(1,n,Root);
128     top=0;dfs(Root);rep(1,n,i)printf("%d ",a[i]);return 0;
129 }

```

```

38 {
39     for (int i = 0; i <= lg; ++i) nw[i] = p[i];
40     for (int i = 0; i <= lg; ++i)
41         for (int j = i - 1; ~j; --j)
42             if (nw[i] >> j & 1) nw[i] ^= nw[j];
43     for (int i = 0; i <= lg; ++i)
44         if (nw[i]) ex.push_back(i);
45 }
46 T get_min_k(T k)
47 {
48     int sz = (int)ex.size();
49     if (sz < n) --k; // element 0
50     if (k > (1ll << sz) - 1) return -1;
51     T ans = 0;
52     for (int j = 0; j < sz; ++j)
53         if (k >> j & 1) ans ^= nw[ex[j]];
54     return ans;
55 }
56 bool find(T x)
57 {
58     for (int i = lg; ~i; --i)
59     {
60         if (!(x >> i & 1)) continue;
61         if (!p[i]) return false;
62         x ^= p[i];
63     }
64     return true;
65 }
66 vector<int> getid(T x) // if find (x) == 1
67 {
68     vector<int> res(n + 1, 0);
69     for (int i = lg; ~i; --i)
70     {
71         if (!(x >> i & 1)) continue;
72         x ^= p[i];
73         for (int j = 1; j <= n; ++j) res[j] ^= id[i][j];
74     }
75     return res;
76 }
77 };

```

2.11 Linear-Basis

```

1 template <typename T>
2 class Basis
3 {
4     int n, lg;
5     vector<T> p, ex, nw; vector<vector<int>>> id;
6     public :
7     Basis(int n, int lg) : n(n), lg(lg), p(lg + 1, 0), nw(lg + 1, 0), id
      (lg + 1) {}
8     void modify(T x, int idx)
9     {
10         vector<int> cur(n + 1, 0); cur[idx] = 1;
11         for (int i = lg; ~i; --i)
12         {
13             if (!(1ll << i) & x) continue;
14             if (!p[i]) {p[i] = x; id[i] = cur; break;}
15             else
16             {
17                 x ^= p[i];
18                 for (int j = 1; j <= n; ++j) cur[j] ^= id[i][j];
19             }
20         }
21     }
22     T get_max()
23     {
24         T ans = 0;
25         for (int i = lg; ~i; --i)
26             if ((ans ^ p[i]) > ans) ans ^= p[i];
27         return ans;
28     }
29     T get_min()
30     {
31         T ans = 0; int cnt = 0;
32         for (int i = 0; i <= lg; ++i) cnt += p[i] > 0;
33         if (cnt < n) return 0;
34         for (int i = 0; i <= lg; ++i)
35             if (p[i] > 0) return p[i];
36     }
37     void change()

```

3 Math

3.1 Subset Enumeration

```
1 for (int S = 1; S < (1 << n); ++S)
2   for (int S0 = S; S0; S0 = (S0 - 1) & S)
```

3.2 Elementary Arithmetic Bucket

```
1 template <typename T, int MOD = 1000000007>
2 class Z
3 {
4     T x;
5     Z <T, MOD> qpow(Z x, T y)
6     {
7         Z <T, MOD> res(1);
8         while (y)
9         {
10             if (y & 1) res *= x;
11             x *= x;
12             y >>= 1;
13         }
14         return res;
15     }
16     public :
17     Z() : x(0) {}
18     Z(T x) : x(x) {}
19     T get() {return x;}
20     Z & operator = (T o) {x = o % MOD; return *this;}
21     Z & operator += (Z o) {x = (x + o.x + MOD) % MOD; return *this;}
22     Z & operator -= (Z o) {x = (x - o.x + MOD) % MOD; return *this;}
23     Z & operator *= (Z o) {x = 1ll * o.x * x % MOD; return *this;}
24     Z & operator ^= (Z o) {x = qpow(x, o.x).x; return *this;}
25     Z & operator /= (Z o) {*this *= o ^ (MOD - 2); return *this;}
26     Z & operator %= (Z o) {x = x % o.x; return *this;}
27     friend Z operator + (Z x, Z y) {return x + y;}
28     friend Z operator - (Z x, Z y) {return x - y;}
29     friend Z operator * (Z x, Z y) {return x * y;}
30     friend Z operator / (Z x, Z y) {return x / y;}
31     friend Z operator ^ (Z x, Z y) {return x ^ y;}
32     friend Z operator % (Z x, Z y) {return x % y;}
33     friend bool operator == (Z x, Z y) {return x.x == y.x;}
34     friend bool operator != (Z x, Z y) {return x.x != y.x;}
35     friend istream& operator >> (istream &is, Z& o) {T val; is >> val; o = Z(val); return is;}
36     friend ostream& operator << (ostream &os, const Z &z) {return os << z.x;}
37 };
```

3.3 Fraction

```
1 template <typename T>
2 class Frac
3 {
4     T p, q; // p / q
5     void reduce()
6     {
7         assert(q != 0);
8         T g = gcd(p, q);
9         p /= g, q /= g;
10        if (q < 0) p = -p, q = -q;
11    }
12    public :
13    Frac(T p = 0, T q = 1) : p(p), q(q) {assert(q != 0); reduce();}
14    T get_p() {return p;}
15    T get_q() {return q;}
16    double get_val() {return 1.0 * p / q;}
17    Frac operator + (const Frac &o) {return Frac(p * o.q + o.p * q, q * o.q);}
18    Frac operator - (const Frac &o) {return Frac(p * o.q - o.p * q, q * o.q);}
19    Frac operator * (const Frac &o) {return Frac(p * o.p, q * o.q);}
20    Frac operator / (const Frac &o) {return Frac(p * o.q, q * o.p);}
21    Frac& operator += (const Frac &o) {*this = *this + o; return *this;}
22    Frac& operator -= (const Frac &o) {*this = *this - o; return *this;}
23    Frac& operator *= (const Frac &o) {*this = *this * o; return *this;}
24    Frac& operator /= (const Frac &o) {*this = *this / o; return *this;}
25    Frac operator ~() {return Frac(-p, q);}
26    bool operator >= (const Frac &o) {return p * o.q >= o.p * q;}
27    bool operator > (const Frac &o) {return p * o.q > o.p * q;}
28    bool operator == (const Frac &o) {return p * o.q == o.p * q;}
29    bool operator <= (const Frac &o) {return !(*this > o);}
30    bool operator < (const Frac &o) {return !(*this >= o);}
31    bool operator != (const Frac &o) {return !(*this == o);}
32 };
```

3.4 Combination

```
1 template <typename T, int MOD = 1000000007>
2 class COM
3 {
4     int n;
5     vector <Z <T, MOD>> fac, inv;
6     public :
7     COM(int n) : n(n), fac(2 * n + 2), inv(2 * n + 2)
8     {
9         fac[0] = inv[0] = inv[1] = 1;
10        for (int i = 1; i <= 2 * n; ++i) fac[i] = fac[i - 1] * i;
11        inv[2 * n] = fac[2 * n] ^ (MOD - 2);
12        for (int i = 2 * n - 1; i > 0; --i) inv[i] = inv[i + 1] * (i + 1);
13    }
14    Z <T, MOD> f(int x) {return fac[x];}
15    Z <T, MOD> inv_f(int x) {return inv[x];}
16    Z <T, MOD> comb(int x, int y)
17    {
18        if (x < 0 || y < 0 || y > x) return 0;
19        else return fac[x] * inv[x - y] * inv[y];
20    }
21    Z <T, MOD> arr(int x, int y)
22    {
23        if (x < 0 || y < 0 || y > x) return 0;
24        else return fac[x] * inv[x - y];
25    }
26    Z <T, MOD> catalan(int x) {return comb(2 * x, x) / (x + 1);}
27 };
```

3.5 The Sieve of Primes & Euler's Totient Function & Möbius Function

```
1 class Prime
2 {
3     int mx;
4     vector <int> p, phi, mu;
5     void pre()
6     {
7         vector <int> fl(mx + 1, 0);
8         phi[1] = mu[1] = 1;
9         for (int i = 2; i <= mx; ++i)
10        {
11            if (!fl[i]) p.push_back(i), phi[i] = i - 1, mu[i] = -1;
12            for (auto v : p)
13            {
14                if (i * v > mx) break;
15                fl[i * v] = 1;
16                if (i % v == 0) {phi[i * v] = phi[i] * v, mu[i * v] = 0; break;}
17                phi[i * v] = phi[i] * (v - 1); mu[i * v] = -mu[i];
18            }
19        }
20    }
21    public :
22    Prime(int mx) : mx(mx), phi(mx + 1, 0), mu(mx + 1, 0) {pre();}
23    auto get_prime() {return p;}
24    auto get_phi() {return phi;}
25    auto get_mu() {return mu;}
26 };
```

3.6 Exgcd

```
1 ll exgcd(ll a, ll b, ll &x, ll &y)
2 {
3     if (!b) {x = 1; y = 0; return abs(a);}
4     ll d = exgcd(b, (a % b + b) % b, y, x);
5     y -= a / b * x;
6     return d;
7 }
```

3.7 CRT

```
1 ll CRT()
2 {
3     //ans % a[i] = b[i] if gcd(a[i], a[j]) = 1
4     ll sum = 1, ans = 0;
5     for (int i = 1; i <= n; ++i) sum *= a[i];
6     for (int i = 1; i <= n; ++i)
7     {
8         ll x, y, tmp = sum / a[i];
9         exgcd(tmp, a[i], x, y);
10        ans = (ans + tmp * x * b[i]) % sum;
11    }
```

```

12     return (ans + sum) % sum;
13 }

```

3.8 Lagrange Interpolation

```

1 ll Lagrange(ll num)
2 {
3     ll ans = 0;
4     for (int i = 1; i <= n; ++i)
5     {
6         ll p = y[i] % MOD, q = 1;
7         for (int j = 1; j <= n; ++j)
8         {
9             if (i == j) continue;
10            p = p * (num - x[j]) % MOD;
11            q = q * (x[i] - x[j]) % MOD;
12        }
13        ans = (ans + p * qpow(q, MOD - 2) % MOD + MOD) % MOD;
14    }
15    return ans;
16 }

```

3.9 Gaussian Elimination

```

1 vector<vector<double>> a(n + 1, vector<double>(n + 2)); // n * (n +
2 1)
3 for (int i = 1; i <= n; ++i)
4 {
5     int p = cur, ok = 0;
6     while (p <= n)
7     {
8         if (a[p][i] != 0) {ok = 1; break;}
9         ++p;
10    }
11    if (!ok) continue;
12    for (int j = i; j <= n + 1; ++j) swap(a[p][j], a[cur][j]);
13    for (int j = n + 1; j >= i; --j) a[cur][j] /= a[cur][i];
14    for (int j = 1; j <= n; ++j)
15    {
16        if (j == cur) continue;
17        for (int k = n + 1; k >= i; --k) a[j][k] -= a[cur][k] * a[j][i];
18    }
19    ++cur;
20 }
21 if (cur <= n) puts("No Solution"); //0 solution or infinte solutions
22 else {for (int i = 1; i <= n; ++i) printf("%.6lf\n", a[i][n + 1]);}

```

3.10 FFT

```

1 using cd = complex<double>;
2 class FFT
3 {
4     const double Pi = acos(-1);
5     vector<int> rev; int n = 1;
6     void fft(vector<cd> &a, int op)
7     {
8         for (int i = 0; i < n; ++i)
9             if (i < rev[i]) swap(a[i], a[rev[i]]);
10        for (int len = 1; len < n; len <= 1)
11        {
12            cd wn(cos(Pi / len), op * sin(Pi / len));
13            for (int i = 0; i < n; i += len <= 1)
14            {
15                cd w(1);
16                for (int j = 0; j < len; ++j)
17                {
18                    cd u = a[i + j], v = a[i + j + len] * w;
19                    a[i + j] = u + v; a[i + j + len] = u - v;
20                    w *= wn;
21                }
22            }
23        }
24        if (op == -1) {for (auto &x : a) x /= n;}
25    }
26    public:
27    vector<double> conv(vector<cd> a, vector<cd> b)
28    {
29        int L = 0, tot = (int)a.size() + (int)b.size() - 1; n = 1;
30        while (n < tot) n <= 1, ++L;
31        rev.resize(n); a.resize(n); b.resize(n);
32        for (int i = 0; i < n; ++i) rev[i] = (rev[i >> 1] >> 1) | ((i &
33 1) << (L - 1));
34        fft(a, 1); fft(b, 1);
35        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i];
36        fft(a, -1);
37        vector<double> ans(tot);
38        for (int i = 0; i < tot; ++i) ans[i] = a[i].real();

```

```

38     return ans;
39 }
40 };

```

3.11 NTT

```

1 template<int MOD = 998244353>
2 class NTT
3 {
4     vector<int> rev; int n = 1;
5     const int P = MOD; //998244353 469762049 50000000001507329LL 4179340
6     454199820289
7     const int G = 3;
8     //1945555039024054273 for G = 5
9     int qpow(int x, int y)
10    {
11        int res = 1;
12        while (y)
13        {
14            if (y & 1) res = 1ll * res * x % P;
15            x = 1ll * x * x % P;
16            y >>= 1;
17        }
18        return res;
19    }
20    void init_rev(int tot)
21    {
22        int L = 0; n = 1;
23        while (n < tot) n <= 1, ++L;
24        rev.resize(n);
25        for (int i = 0; i < n; ++i) rev[i] = (rev[i >> 1] >> 1) | ((i &
26 1) << (L - 1));
27    }
28    void ntt(vector<int> &a, int op)
29    {
30        for (int i = 0; i < n; ++i)
31            if (i < rev[i]) swap(a[i], a[rev[i]]);
32        for (int len = 1; len < n; len <= 1)
33        {
34            int wn = qpow(G, (P - 1) / (len <= 1));
35            if (op == -1) wn = qpow(wn, P - 2);
36            for (int i = 0; i < n; i += len <= 1)
37            {
38                int w = 1;
39                for (int j = 0; j < len; ++j)
40                {
41                    int u = a[i + j] >= P ? a[i + j] - P : a[i + j], v =
42                    1ll * a[i + j + len] * w % P;
43                    a[i + j] = u + v >= P ? u + v - P : u + v;
44                    a[i + j + len] = u - v <= 0 ? u - v + P : u - v;
45                    w = 1ll * w * wn % P;
46                }
47            }
48            if (op == -1)
49            {
50                int inv_n = qpow(n, P - 2);
51                for (auto &x : a) x = 1ll * x * inv_n % P;
52            }
53        }
54    }
55    public:
56    vector<int> Conv(vector<int> a, vector<int> b)
57    {
58        int tot = (int)a.size() + (int)b.size() - 1;
59        init_rev(tot);
60        a.resize(n); b.resize(n);
61        ntt(a, 1); ntt(b, 1);
62        for (int i = 0; i < n; ++i) a[i] = 1ll * a[i] * b[i] % P;
63        ntt(a, -1); a.resize(tot);
64        return a;
65    }
66    vector<int> Inv(vector<int> a, int m)
67    {
68        int inv = qpow(a[0], P - 2);
69        vector<int> b(inv);
70        for (int len = 2; (len >> 1) < m; len <= 1)
71        {
72            int sz = min((int)a.size(), len);
73            vector<int> f(a.begin(), a.begin() + sz);
74            init_rev(len <= 1);
75            f.resize(n);
76            ntt(f, 1); ntt(h, 1);
77            for (int i = 0; i < n; ++i)
78                h[i] = 1ll * h[i] * (2ll - 1ll * f[i] * h[i] % P + P) %
79                P;
80            ntt(h, -1);
81            b.assign(h.begin(), h.begin() + min(len, m));
82        }
83        b.resize(m);
84        return b;
85    }

```

```

83 pair <vector <int>, vector <int>> Divmod(vector <int> a, vector <int>
84 b)
85 {
86     int n_sz = a.size(), m_sz = b.size();
87     if (n_sz < m_sz) return {{0}, a};
88     int dq = n_sz - m_sz + 1;
89     vector <int> ra = a, rb = b;
90     reverse(ra.begin(), ra.end()); reverse(rb.begin(), rb.end());
91     ra.resize(dq); rb.resize(dq);
92     vector <int> inv_rb = Inv(rb, dq);
93     vector <int> q = Conv(ra, inv_rb);
94     q.resize(dq);
95     reverse(q.begin(), q.end());
96     vector <int> qb = Conv(q, b);
97     vector <int> r(m_sz - 1);
98     for (int i = 0; i < m_sz - 1; ++i)
99         r[i] = (0ll + a[i] - (i < (int)qb.size() ? qb[i] : 0) + P) %
100         P;
101     return {q, r};
102 }
103 vector <int> Sqrt(vector <int> a, int m) // b[0] = 1
104 {
105     vector <int> b{1};
106     int inv2 = qpow(2, P - 2);
107     for (int len = 2; (len >> 1) < m; len <= 1)
108     {
109         int sz = min((int)a.size(), len);
110         vector <int> f(a.begin(), a.begin() + sz);
111         vector <int> inv_b = Inv(b, len);
112         init_rev(len <= 1);
113         f.resize(n); inv_b.resize(n);
114         ntt(f, 1); ntt(inv_b, 1);
115         for (int i = 0; i < n; ++i)
116             f[i] = 1ll * f[i] * inv_b[i] % P;
117         ntt(f, -1);
118         b.resize(len);
119         for (int i = 0; i < len; ++i)
120             b[i] = 1ll * (b[i] + f[i]) % P * inv2 % P;
121     }
122     b.resize(m);
123     return b;
124 };

```

```

45 }
46 vector <T> mul(vector <T> A, vector <T> B)
47 {
48     int n = A.size();
49     vector <T> C(n);
50     for (int i = 0; i < n; ++i) C[i] = A[i] * B[i] % P;
51     return C;
52 }
53 public :
54 FWT() {}
55 vector <T> calc_Or(vector <T> A, vector <T> B)
56 {
57     Or(A); Or(B);
58     auto C = mul(A, B);
59     Or(C, P - 1);
60     return C;
61 }
62 vector <T> calc_And(vector <T> A, vector <T> B)
63 {
64     And(A); And(B);
65     auto C = mul(A, B);
66     And(C, P - 1);
67     return C;
68 }
69 vector <T> calc_Xor(vector <T> A, vector <T> B)
70 {
71     Xor(A); Xor(B);
72     auto C = mul(A, B);
73     Xor(C, inv_2);
74     return C;
75 }
76 };

```

3.12 FWT

```

1 template <typename T>
2 class FWT
3 {
4     public:
5     const int P = 998244353;
6     const ll inv_2 = 499122177; //500000004 for 1e9 + 7
7     private:
8     void Or(vector <T> &F, int op = 1)
9     {
10         int n = F.size();
11         for (int i = 2; i <= n; i <= 1)
12         {
13             int o = i >> 1;
14             for (int j = 0; j < n; j += i)
15                 for (int k = 0; k < o; ++k) F[j + k + o] = (F[j + k + o]
16                 + F[j + k] * op % P) % P;
17         }
18     }
19     void And(vector <T> &F, int op = 1)
20     {
21         int n = F.size();
22         for (int i = 2; i <= n; i <= 1)
23         {
24             int o = i >> 1;
25             for (int j = 0; j < n; j += i)
26                 for (int k = 0; k < o; ++k) F[j + k] = (F[j + k] + F[j +
27                 k + o] * op % P) % P;
28         }
29     }
30     void Xor(vector <T> &F, int op = 1)
31     {
32         int n = F.size();
33         for (int i = 2; i <= n; i <= 1)
34         {
35             int o = i >> 1;
36             for (int j = 0; j < n; j += i)
37             {
38                 for (int k = 0; k < o; ++k)
39                 {
40                     F[j + k] = (F[j + k] + F[j + k + o]) % P;
41                     F[j + k + o] = (F[j + k] - 2 * F[j + k + o] % P + P)
42                     % P;
43                     F[j + k] = F[j + k] * op % P;
44                     F[j + k + o] = F[j + k + o] * op % P;
45                 }
46             }
47         }
48     }
49 };

```

4 Computational Geometry

```

1 using LD = long double;
2 const LD pi = acos(-1.0);
3 const LD eps = 1e-8L;
4 int dcmp(LD x) {return x < -eps ? -1 : (x > eps ? 1 : 0);}
5 struct Point {LD x, y; Point(LD x = 0, LD y = 0) : x(x), y(y) {};}
6 struct Circle {Point O; LD r; Circle(Point O = Point(), LD r = 0) : O(O),
7   r(r) {};}
8 typedef Point Vector;
9 Vector operator + (Vector A, Vector B) {return Vector(A.x + B.x, A.y + B.y);}
10 Vector operator - (Vector A, Vector B) {return Vector(A.x - B.x, A.y - B.y);}
11 Vector operator * (Vector A, LD k) {return Vector(A.x * k, A.y * k);}
12 Vector operator / (Vector A, LD k) {return Vector(A.x / k, A.y / k);}
13 bool operator == (Vector A, Vector B) {return dcmp(A.x - B.x) == 0 &&
14   dcmp(A.y - B.y) == 0;}
15 bool operator != (Vector A, Vector B) {return !(A == B);}
16 LD dot(Vector A, Vector B) {return A.x * B.x + A.y * B.y;}
17 LD dis(Point A, Point B) {return sqrt((A.x - B.x) * (A.x - B.x) + (A.y -
18   B.y) * (A.y - B.y));}
19 LD cross(Vector A, Vector B) {return A.x * B.y - A.y * B.x;} // A -> B
20   counter-clockwise if cross(A, B) > 0
21 LD len(Point A) {return sqrt(A.x * A.x + A.y * A.y);}
22 LD angle(Vector A, Vector B) {return acos(dot(A, B) / (len(A) * len(B)))
23   ;}
24 Vector proj(Vector A, Vector B) {return A * (dot(A, B) / dot(A, A));} //
25   project onto A
26 Point foot(Point P, Point A, Point B) {Vector AP = P - A, AB = B - A;
27   return A + proj(AB, AP);} // foot
28 Point reflect(Point P, Point A, Point B) {Point F = foot(P, A, B);
29   return F * 2 - P;} // symmetry point
30 Point rotate(Point P, LD theta) {return (Point){P.x * cos(theta) - P.y *
31   sin(theta), P.x * sin(theta) + P.y * cos(theta)}}; //
32   counterclockwise
33 bool chk_on_line(Point P, Point A, Point B) {return dcmp(cross(P - A, B -
34   A)) == 0;} // judge whether on line AB
35 bool chk_on_seg(Point P, Point A, Point B) {return chk_on_line(P, A, B)
36   && dcmp(dot(P - A, P - B)) <= 0;} // judge whether on segment AB
37 pair <Point, Point> get_parallel(Point P, Point A, Point B) {return {P,
38   P + (B - A)}; // the line passing through P and parallel to line AB
39 pair <Point, Point> get_vertical(Point P, Point A, Point B) // the line
40   passing through P and perpendicular to line AB
41 {
42   Vector V = B - A, N = {-V.y, V.x};
43   return {P, P + N};
44 }
45 LD dis_seg(Point P, Point A, Point B) // the distance from point P to
46   segment AB.
47 {
48   if (dcmp(dot(B - A, P - A)) < 0) return dis(P, A);
49   if (dcmp(dot(A - B, P - B)) < 0) return dis(P, B);
50   return fabs(cross(P - A, P - B)) / dis(A, B);
51 }
52 Point ll_inter_pt(Point A, Point B, Point C, Point D) {return A + (B - A)
53   * cross(C - A, D - C) / cross(B - A, D - C);}
54 bool chk_ll_inter(Point A, Point B, Point C, Point D) {return dcmp(cross
55   (B - A, D - C)) != 0;} // line - line
56 bool chk_ls_inter(Point A, Point B, Point C, Point D) {return chk_on_seg
57   (ll_inter_pt(A, B, C, D), C, D);} // The intersection of AB(line)
58   and CD (line) is on the CD (seg).
59 bool chk_ss_inter(Point A, Point B, Point C, Point D) // seg - seg
60 {
61   LD c1 = cross(B - A, C - A), c2 = cross(B - A, D - A);
62   LD d1 = cross(D - C, A - C), d2 = cross(D - C, B - C);
63   if (dcmp(c1) * dcmp(c2) < 0 && dcmp(d1) * dcmp(d2) < 0) return true;
64   if (dcmp(c1) == 0 && chk_on_seg(C, A, B)) return true;
65   if (dcmp(c2) == 0 && chk_on_seg(D, A, B)) return true;
66   if (dcmp(d1) == 0 && chk_on_seg(A, C, D)) return true;
67   if (dcmp(d2) == 0 && chk_on_seg(B, C, D)) return true;
68   return false;
69 }
70 bool SameSide(Point A, Point B, Point C, Point D) {return cross(A - C, D
71   - C) * cross(D - C, B - C) < 0;} // Point A, B Line C, D
72 LD area(vector <Point> &P)
73 {
74   int n = P.size();
75   LD res = 0;
76   for (int i = 0; i < n; ++i) res += cross(P[i], P[(i + 1) % n]);
77   return res / 2.0;
78 }
79 bool is_convex(vector <Point> &P)
80 {
81   int n = P.size();
82   for (int i = 0; i < n - 1; ++i)
83     if (dcmp(cross(P[i + 1] - P[i], P[(i + 2) % n] - P[i])) < 0)
84       return false;
85   return true;
86 }
87 int in_Poly(vector <Point> &P, Point A) // 0 (n) for any polygons
88 {

```

```

70 int cnt = 0, n = P.size();
71 for (int i = 0; i < n; ++i)
72 {
73   int j = (i + 1) % n;
74   if (chk_on_seg(A, P[i], P[j])) return 2; // on the edge
75   if (A.y >= min(P[i].y, P[j].y) && A.y < max(P[i].y, P[j].y)) //
76     the intersection is on the right
77     cnt += dcmp((A.y - P[i].y) * (P[j].x - P[i].x) / (P[j].y -
78       P[i].y) + P[i].x - A.x) > 0;
79 }
80 return cnt & 1;
81 }
82 int in_convex_Poly(vector <Point> &P, Point A) // 0 (log n) only for
83   convex polygons
84 {
85   int n = P.size();
86   if (chk_on_seg(A, P[0], P[1]) || chk_on_seg(A, P[0], P[n - 1]))
87     return 2;
88   if (dcmp(cross(P[n - 1] - P[0], A - P[0])) > 0 || dcmp(cross(P[1] -
89     P[0], A - P[0])) < 0) return 0;
90   int l = 1, r = n - 2, res = -1;
91   while (l <= r)
92   {
93     int mid = (l + r) >> 1;
94     if (dcmp(cross(P[mid] - P[0], A - P[0])) >= 0) res = mid, l =
95       mid + 1;
96     else r = mid - 1;
97   }
98   if (chk_on_seg(A, P[res], P[res + 1])) return 2;
99   if (dcmp(cross(P[res + 1] - P[res], A - P[res])) < 0) return 0;
100   return 1;
101 }
102 auto convex_hull(vector <Point> &P) // strict convex hull (<= 0)
103 {
104   int n = P.size();
105   sort(P.begin(), P.end(), [](Point &x, Point &y) {return x.x == y.x
106     ? x.y < y.y : x.x < y.x;});
107   vector <Point> hull;
108   hull.resize(2 * n + 1);
109   int k = 0;
110   for (int i = 0; i < n; ++i)
111   {
112     while (k >= 2 && dcmp(cross(hull[k - 1] - hull[k - 2], P[i] -
113       hull[k - 2])) <= 0) --k;
114     hull[k++] = P[i];
115   }
116   for (int i = n - 2, t = k; i >= 0; --i)
117   {
118     while (k > t && dcmp(cross(hull[k - 1] - hull[k - 2], P[i] -
119       hull[k - 2])) <= 0) --k;
120     hull[k++] = P[i];
121   }
122   hull.resize(k - 1);
123   return hull;
124 }
125 LD diameter(vector <Point> &P)
126 {
127   int n = P.size();
128   if (n <= 1) return 0;
129   if (n == 2) return len(P[1] - P[0]);
130   LD res = 0;
131   for (int i = 0, j = 2; i < n; ++i)
132   {
133     while (dcmp(cross(P[(i + 1) % n] - P[i], P[j] - P[i]) - cross(P
134       [(i + 1) % n] - P[i], P[(j + 1) % n] - P[i])) <= 0) j = (j + 1) %
135       n;
136     res = max(res, max(len(P[j] - P[i]), len(P[j] - P[(i + 1) % n])));
137   }
138   return res;
139 }
140 template <typename Line = pair <Point, Point>>
141 LD half_plane(vector <Line> &Vec)
142 {
143   int n = Vec.size();
144   auto get_angle = [&] (Line line) -> LD {return atan2(line.second.y -
145     line.first.y, line.second.x - line.first.x);};
146   sort(Vec.begin(), Vec.end(), [&] (Line A, Line B)
147   {
148     LD angA = get_angle(A), angB = get_angle(B);
149     return fabs(angA - angB) > eps ? angA < angB : cross(A.second -
150       A.first, B.second - A.first) < 0;
151   });
152   int h = 1, t = 0;
153   auto check = [&] (Line x, Line y, Line z) -> bool
154   {
155     Point P = ll_inter_pt(y.first, y.second, z.first, z.second);
156     return dcmp(cross(x.second - x.first, P - x.first)) < 0;
157   };
158   vector <Line> q(n + 10); q[0] = Vec[0];
159   for (int i = 1; i < n; ++i)
160   {
161     if (get_angle(Vec[i]) - get_angle(Vec[i - 1]) < eps) continue;
162     while (h < t && check(Vec[i], q[t], q[t - 1])) --t;
163     while (h < t && check(Vec[i], q[h], q[h + 1])) ++h;
164     q[t++] = Vec[i];
165   }

```



```

152 }
153 while (h < t && check(q[h], q[t], q[t - 1])) --t;
154 q[++t] = q[h];
155 vector <Point> p;
156 for (int i = h; i < t; ++i) p.push_back(ll_inter_pt(q[i].first, q[i].
157 second, q[i + 1].first, q[i + 1].second));
158 return area(p);
159 }
160 vector <Point> minkowski(vector <Point> &vecA, vector <Point> &vecB)
161 {
162     int n = vecA.size(), m = vecB.size();
163     vector <Point> A(n), B(m), C {vecA[0] + vecB[0]};
164     for (int i = 0; i < n; ++i) A[i] = vecA[(i + 1) % n] - vecA[i];
165     for (int i = 0; i < m; ++i) B[i] = vecB[(i + 1) % m] - vecB[i];
166     int posa = 0, posb = 0;
167     while (posa < n || posb < m)
168     {
169         if (posa == n) C.push_back(C.back() + B[posb++]);
170         else if (posb == m) C.push_back(C.back() + A[posa++]);
171         else if (dcmp(cross(A[posa], B[posb])) >= 0) C.push_back(C.back()
172 + A[posa++]);
173         else C.push_back(C.back() + B[posb++]);
174     }
175     return convex_hull(C);
176 }
177 bool chk_convex_inter_strict(vector <Point> A, vector <Point> B) //
178 Check if two convex polygons strictly intersect
179 {
180     vector <Point> negB;
181     for (auto p : B) negB.push_back(Point(-p.x, -p.y));
182     negB = convex_hull(negB);
183     vector <Point> C = minkowski(A, negB);
184     if (C.size() < 3) return false;
185     Point origin(0, 0);
186     return in_convex_Poly(C, origin) == 1;
187 }
188 bool chk_in_cir(Circle C, Point P) {return dcmp(len(P - C.O) - C.r) <= 0
189 ;}
190 Point get_cir_pt(Circle C, LD theta) {return {C.O.x + C.r * cos(theta),
191 C.O.y + C.r * sin(theta)};}
192 int chk_lc_inter(Point A, Point B, Circle C)
193 {
194     LD d = dis_seg(C.O, A, B);
195     if (dcmp(d - C.r) == 0) return 0; // tangent
196     if (dcmp(d - C.r) > 0) return -1; // separation
197     return 1; // intersection
198 }
199 int chk_cc_inter(Circle A, Circle B) // the number of tagent lines
200 {
201     LD d = len(A.O - B.O);
202     if (dcmp(A.r + B.r - d) < 0) return 4; // externally separate
203     if (dcmp(A.r + B.r - d) == 0) return 3; // externally tangent
204     if (dcmp(fabs(A.r - B.r) - d) == 0) return 1; // internally tangent
205     if (dcmp(fabs(A.r - B.r) - d) > 0) return 0; // one circle inside
206     the other
207     return 2; // intersection
208 }
209 pair <Point, Point> lc_inter(Point A, Point B, Circle C)
210 {
211     Point F = foot(C.O, A, B); LD d = dis(C.O, F);
212     Vector E = (B - A) / dis(A, B);
213     Point P1 = F - E * sqrt(C.r * C.r - d * d);
214     Point P2 = F + E * sqrt(C.r * C.r - d * d);
215     return {P1, P2};
216 }
217 pair <Point, Point> cc_inter(Circle A, Circle B)
218 {
219     Vector k = B.O - A.O;
220     LD d = len(k);
221     LD alpha = atan2(k.y, k.x), beta = acos((A.r * A.r + d * d - B.r * B.
222 r) / (2 * A.r * d));
223     Point P1 = get_cir_pt(A, alpha - beta), P2 = get_cir_pt(A, alpha +
224 beta);
225     return {P1, P2};
226 }
227 pair <Point, Point> tan_cir(Point P, Circle C)
228 {
229     LD d = len(C.O - P), theta = asin(C.r / d);
230     Vector E = (C.O - P) / d;
231     Vector P1 = P + (rotate(E, theta) * sqrt(d * d - C.r * C.r));
232     Vector P2 = P + (rotate(E, -theta) * sqrt(d * d - C.r * C.r));
233     return {P1, P2};
234 }
235 Circle tri_incir(Point A, Point B, Point C)
236 {
237     LD a = dis(B, C), b = dis(A, C), c = dis(A, B);
238     Point O = (A * a + B * b + C * c) / (a + b + c);
239     return {O, dis_seg(O, A, B)};
240 }
241 Circle tri_circum(Point A, Point B, Point C)
242 {
243     LD Bx = B.x - A.x, By = B.y - A.y, Cx = C.x - A.x, Cy = C.y - A.y;
244     LD D = 2 * (Bx * Cy - By * Cx);
245     LD x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.
246 x;

```

```

239 LD y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.
240 y;
241 Point P(x, y);
242 return Circle(P, dis(A, P));
243 }
244 auto get_tangents(Circle A, Circle B)
245 {
246     vector <pair <Point, Point>> tangents;
247     LD d = len(A.O - B.O), dif = A.r - B.r, sum = A.r + B.r;
248     if (dcmp(d - fabs(dif)) < 0) return tangents;
249     LD base = atan2(B.O.y - A.O.y, B.O.x - A.O.x);
250     if (dcmp(d - fabs(dif)) == 0)
251     {
252         tangents.push_back({get_cir_pt(A, base + (A.r < B.r ? pi : 0)),
253 get_cir_pt(A, base + (A.r < B.r ? pi : 0))});
254         return tangents;
255     }
256     LD theta = acos(dif / d);
257     tangents.push_back({get_cir_pt(A, base + theta), get_cir_pt(B, base +
258 theta)});
259     tangents.push_back({get_cir_pt(A, base - theta), get_cir_pt(B, base -
260 theta)});
261     if (dcmp(d - sum) == 0) tangents.push_back({get_cir_pt(A, base),
262 get_cir_pt(A, base)});
263     if (dcmp(d - sum) > 0)
264     {
265         theta = acos(sum / d);
266         tangents.push_back({get_cir_pt(A, base + theta), get_cir_pt(B,
267 base + theta + pi)});
268         tangents.push_back({get_cir_pt(A, base - theta), get_cir_pt(B,
269 base - theta + pi)});
270     }
271     return tangents;
272 }
273 LD tri_cir_area(Point A, Point B, Circle C)
274 {
275     Vector OA = A - C.O, OB = B - C.O;
276     LD S = cross(OA, OB), sign = dcmp(cross(OA, OB)) > 0 ? 1 : -1;
277     bool da = dcmp(len(OA) - C.r) < 0, db = dcmp(len(OB) - C.r) < 0;
278     if (dcmp(S) == 0) return 0;
279     if (da && db) return S * 0.5; // triangle
280     if (!da && !db)
281     {
282         if (chk_lc_inter(A, B, C) == 1) // arc + triangle + arc
283         {
284             auto [P1, P2] = lc_inter(A, B, C);
285             Vector OP1 = P1 - C.O, OP2 = P2 - C.O;
286             if (dis(A, P1) > dis(A, P2)) swap(P1, P2);
287             return cross(OP1, OP2) * 0.5 + sign * 0.5 * C.r * C.r * (
288 angle(OA, OP1) + angle(OB, OP2));
289         }
290         else return sign * 0.5 * C.r * C.r * angle(OA, OB); // arc
291     }
292     else // triangle + arc
293     {
294         auto [P1, P2] = lc_inter(A, B, C);
295         if (chk_on_seg(P2, A, B)) swap(P1, P2);
296         Vector OP1 = P1 - C.O;
297         if (dcmp(len(OA) - C.r) < 0) return cross(OA, OP1) * 0.5 + sign *
298 0.5 * C.r * C.r * angle(OP1, OB);
299         else return cross(OP1, OB) * 0.5 + sign * 0.5 * C.r * C.r *
300 angle(OP1, OA);
301     }
302 }
303 LD cc_area(Circle C1, Circle C2)
304 {
305     int op = chk_cc_inter(C1, C2);
306     if (op <= 1) return pi * min(C1.r, C2.r) * min(C1.r, C2.r);
307     else if (op == 4) return 0;
308     else
309     {
310         LD d = dis(C1.O, C2.O);
311         LD alpha = 2 * acos((C1.r * C1.r - C2.r * C2.r + d * d) / (2 * C
312 1.r * d));
313         LD beta = 2 * acos((C2.r * C2.r - C1.r * C1.r + d * d) / (2 * C2.
314 r * d));
315         return 0.5 * (C1.r * C1.r * (alpha - sin(alpha)) + C2.r * C2.r *
316 (beta - sin(beta)));
317     }
318 }

```


5 String

5.1 Hash

```

1 template <unsigned long long base = 13331>
2 class Hash
3 {
4     using u64 = unsigned long long;
5     vector <u64> pw, hsh;
6     public:
7     Hash(char *s)
8     {
9         int n = strlen(s);
10        pw.assign(n + 1, 0), hsh.assign(n + 1, 0);
11        pw[0] = 1; hsh[0] = 0;
12        for (int i = 1; i <= n; ++i) pw[i] = pw[i - 1] * base, hsh[i] =
13            hsh[i - 1] * base + s[i - 1];
14    }
15    u64 get(int l, int r) {return hsh[r] - hsh[l - 1] * pw[r - l + 1];}
16    u64 link(int l1, int r1, int l2, int r2) {return get(l1, r1) * pw[r2
17        - l2 + 1] + get(l2, r2);}
18    bool same(int l1, int r1, int l2, int r2) {return get(l1, r1) == get
19        (l2, r2);}
20 };

```

5.2 KMP

```

1 class KMP
2 {
3     vector <int> fail, ans;
4     void getfail(char *s) // Match a with b
5     {
6         int len = strlen(s);
7         fail.assign(len + 1, 0);
8         fail[0] = -1;
9         for (int i = 1; i < len; ++i)
10        {
11            int cnt = fail[i - 1];
12            while (cnt >= 0 && s[cnt + 1] != s[i]) cnt = fail[cnt];
13            if (s[cnt + 1] == s[i]) ++cnt;
14            fail[i] = cnt;
15        }
16    }
17    public:
18    auto get_pos(char *s, char *t)
19    {
20        getfail(t);
21        int lens = strlen(s), lent = strlen(t), cnt = -1;
22        for (int i = 0; i < lens; ++i)
23        {
24            while (cnt >= 0 && t[cnt + 1] != s[i]) cnt = fail[cnt];
25            if (t[cnt + 1] == s[i])
26            {
27                ++cnt;
28                if (cnt + 1 == lent) ans.push_back(i - lent + 2), cnt =
29                    fail[cnt];
30            }
31            return ans;
32        }
33    };

```

5.3 Manacher

```

1 int Manacher(char *s)
2 {
3     int n = strlen(s), cnt = 0, r = 0, mid = 0, ans = 0;
4     vector <char> a(2 * n + 5); vector <int> p(2 * n + 5);
5     a[++cnt] = '!'; a[++cnt] = '#';
6     for (int i = 0; i < n; ++i) a[++cnt] = s[i], a[++cnt] = '#';
7     a[++cnt] = '~';
8     for (int i = 2; i < cnt; ++i)
9     {
10        if (i <= r) p[i] = min(r - i + 1, p[mid * 2 - i]);
11        else p[i] = 1;
12        while (a[i - p[i]] == a[i + p[i]]) ++p[i];
13        if (i + p[i] > r) r = i + p[i] - 1, mid = i;
14        ans = max(ans, p[i]);
15    }
16    return ans - 1;
17 }

```

5.4 Trie

5.4.1 Trie

```

1 struct Trie
2 {
3     int n, m, cnt; //m total len
4     vector <vector <int>> ch; vector <int> vis;
5     public:
6     Trie(int n, int m) : n(n), m(m), ch(m, vector <int> (26, 0)), vis(m)
7     {cnt = 0;}
8     void insert(char *s)
9     {
10        int u = 0, len = strlen(s);
11        for (int i = 0; i < len; ++i)
12        {
13            int c = s[i] - 'a';
14            if (!ch[u][c]) ch[u][c] = ++cnt;
15            u = ch[u][c];
16        }
17        ++vis[u];
18    }
19    int query(char *s)
20    {
21        int u = 0, len = strlen(s);
22        for (int i = 0; i < len; ++i)
23        {
24            int c = s[i] - 'a';
25            if (!ch[u][c]) return 0;
26            u = ch[u][c];
27        }
28        return vis[u];
29    };

```

5.4.2 01-Trie

```

1 class Trie
2 {
3     int n, cnt;
4     vector <vector <int>> ch;
5     vector <int> val, w;
6     public :
7     Trie(int n, int lg) : n(n), val(2 * lg * n + 1, 0), w(2 * lg * n + 1,
8         0), ch(2 * lg * n + 1, vector <int> (2, 0)) {cnt = 1;}
9     void pushup(int u)
10    {
11        w[u] = val[u] = 0;
12        //w[u] Number of values (weights) on the edge between node u and
13        //its parent node
14        //val[u] XOR sum maintained by the subtree rooted at u
15        if (ch[u][0]) w[u] ^= w[ch[u][0]], val[u] ^= val[ch[u][0]] << 1;
16        if (ch[u][1]) w[u] ^= w[ch[u][1]], val[u] ^= (val[ch[u][1]] << 1)
17        | w[ch[u][1]];
18    }
19    void modify(int &u, int v, int dep)
20    {
21        if (!u) u = ++cnt;
22        w[u] ^= 1;
23        if (dep < 0) return ;
24        modify(ch[u][v & 1], v >> 1, dep - 1);
25        pushup(u);
26    }
27    void erase(int u, int v, int dep)
28    {
29        if (!u) return ;
30        w[u] ^= 1;
31        if (dep < 0) return ;
32        erase(ch[u][v & 1], v >> 1, dep - 1);
33        pushup(u);
34    }
35    void add(int u) // add 1 in [1,n]
36    {
37        swap(ch[u][0], ch[u][1]);
38        if (ch[u][0]) add(ch[u][0]);
39        pushup(u);
40    }
41 };

```

5.5 Aho-Corasick Automaton

```

1 class AC
2 {
3     int n, m, cnt; //m total len
4     vector <vector <int>> tr; vector <int> val;
5     public :
6     Trie(int n, int m) : n(n), m(m), tr(m + 1, vector <int> (26, 0)),
7         val(m + 1), fail(m + 1) {cnt = 1;}
8     void insert(char *s)
9     {
10        int n = strlen(s + 1), u = 1;
11        for (int i = 1; i <= N; ++i)
12        {
13            if (!tr[u][s[i] - 'a']) tr[u][s[i] - 'a'] = ++cnt;

```

```

13     u = tr[u][s[i] - 'a'];
14 }
15 ++val[u];
16 }
17 void build()
18 {
19     for (int i = 0; i < 26; ++i) tr[0][i] = 1;
20     queue<int> q; q.push(1);
21     while (!q.empty())
22     {
23         int u = q.front(); q.pop();
24         for (int i = 0; i < 26; ++i)
25         {
26             if (!tr[u][i]) tr[u][i] = tr[fail[u]][i];
27             else fail[tr[u][i]] = tr[fail[u]][i], q.push(tr[u][i]);
28         }
29     }
30 }
31 int query(char *s)
32 {
33     int n = strlen(s + 1), u = 1, res = 0;
34     for (int i = 1; i <= n; ++i)
35     {
36         u = tr[u][s[i] - 'a'];
37         for (int j = u; j && ~val[j]; j = fail[j]) res += val[j],
38         val[j] = -1;
39     }
40     return res;
41 }
42 /*
43 s :
44 a
45 aa
46 aa
47 aaa
48 t :
49 aaaa
50 ans : 4
51 */

```

5.6 SA

```

1 struct SA
2 {
3     string s;
4     vector<int> sa, rk, ht;
5     // all 0-base including sa
6     // suffix order
7     // rank of suffix i
8     // LCP(sf[i], sf[i+1])
9     vector<int> tmp;
10    int n;
11    SA(string &t) {init(t);}
12    inline void tSort(int m) //size of char set (base sort)
13    {
14        vector<int> tmp1(m + 1, 0);
15        for (int i = 0; i < n; ++i) ++tmp1[rk[tmp[i]]];
16        for (int i = 1; i <= m; ++i) tmp1[i] += tmp1[i - 1];
17        for (int i = n - 1; i >= 0; --i) sa[--tmp1[rk[tmp[i]]]] = tmp[i];
18    }
19    inline void init(string &t)
20    {
21        s = t; n = s.size();
22        sa.assign(n, 0); rk.assign(n, 0); ht.assign(n, 0); tmp.assign(n, 0);
23        for (int i = 0; i < n; ++i) rk[i] = s[i], tmp[i] = i;
24        int m = 127; tSort(m);
25        for (int k = 1, p = 0; p < n; k <= 1)
26        {
27            p = 0;
28            for (int i = n - k; i < n; ++i) tmp[p++] = i;
29            for (int i = 0; i < n; ++i)
30                if (sa[i] >= k) tmp[p++] = sa[i] - k;
31            tSort(m);
32            vector<int> old_rk = rk;
33            rk[sa[0]] = p = 1;
34            for (int i = 1; i < n; ++i)
35            {
36                int a = sa[i], b = sa[i - 1];
37                if (old_rk[a] == old_rk[b] && (a + k < n ? old_rk[a + k]
38                : 0) == (b + k < n ? old_rk[b + k] : 0))
39                    rk[a] = p;
40                else
41                    rk[a] = ++p;
42            }
43            m = p;
44            if (p == n) break;
45        }
46        for (int i = 0; i < n; ++i) --rk[i];
47        int h = 0;
48        for (int i = 0; i < n; ++i)

```

```

48    {
49        if (rk[i] == 0) continue;
50        int j = sa[rk[i] - 1];
51        while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
52        ht[rk[i] - 1] = h;
53        if (h > 0) --h;
54    }
55 }
56 inline void clear()
57 {
58     s.clear(); sa.clear(); rk.clear(); ht.clear(); tmp.clear();
59     n = 0;
60 }
61 inline int size() const {return n;}
62 inline int operator[] (int p) const {return sa[p];}
63 };

```

5.7 SAM

```

1 class SAM
2 {
3     class node
4     {
5     public: int ch[26], len, fa;
6     node(const int &L = 0) {memset(ch, 0, sizeof(ch)); fa = 0; len
7     = L;}
8     };
9     public:
10    vector<node> t; int lst;
11    void GetParentTree(vector<vector<int>> &G)
12    {
13        G.assign(t.size(), vector<int>());
14        for (unsigned i = 1; i < t.size(); ++i) G[t[i].fa].push_back(i);
15    }
16    void extend(const int &c)
17    {
18        int p = lst, np = t.size();
19        t.push_back(node(t[p].len + 1));
20        for (; p && t[p].ch[c]; p = t[p].fa) t[p].ch[c] = np;
21        if (!p) t[np].fa = 1;
22        else
23        {
24            int v = t[p].ch[c];
25            if (t[v].len == t[p].len + 1) t[np].fa = v;
26            else
27            {
28                int nv = t.size(); t.push_back(t[v]);
29                t[nv].len = t[p].len + 1;
30                for (; p && t[p].ch[c] == v; p = t[p].fa) t[p].ch[c] =
31                nv;
32                t[np].fa = t[v].fa = nv;
33            }
34        }
35    }
36    SAM() {t.assign(2, node()); lst = 1;}
37    inline void clear() {t.assign(2, node()); lst = 1;}
38    inline int next(int p, int c) {return t[p].ch[c];}
39    inline int Len(int p) {return t[p].len;}
40 };

```

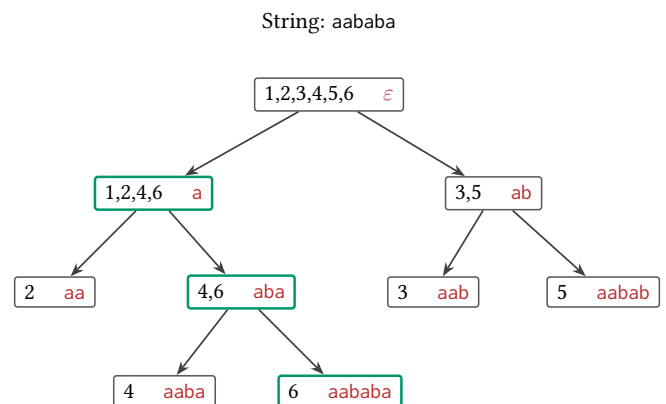


Figure 1: Suffix-link tree for the suffix automaton of aababa; green boxes mark terminal states.

6 Other

6.1 Fast I/O

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 constexpr int MAX = 1e5 + 5;
4 constexpr int MOD = 1e9 + 7;
5 constexpr int INF = 0x3f3f3f3f;
6 using i64 = long long;
7 using u64 = unsigned long long;
8 using pii = pair<int, int>;
9 using vi = vector<int>;
10 using vii = vector<pii>;
11 using LD = long double;
12 inline int read()
13 {
14     int s = 0; int f = 1;
15     char ch = getchar();
16     while ((ch < '0' || ch > '9') && ch != EOF)
17     {
18         if (ch == '-') f = -1;
19         ch = getchar();
20     }
21     while (ch >= '0' && ch <= '9')
22     {
23         s = s * 10 + ch - '0';
24         ch = getchar();
25     }
26     return s * f;
27 }
28 inline void write(int x)
29 {
30     if (x < 0) {putchar('-'); write(-x); return;}
31     if (x > 9) write(x / 10);
32     putchar(x % 10 + '0');
33 }
34 int main()
35 {
36     //freopen("in", "r", stdin);
37     //freopen("out", "w", stdout);
38     return 0;
39 }

```

6.2 Random Number Generator

```

1 int rd(int a, int b)
2 {
3     static std::mt19937 gen(std::random_device{}());
4     std::uniform_int_distribution<int> dist(a, b);

```

```

5     return dist(gen);
6 }

```

6.3 Execution

```

1 #!/bin/bash
2 ulimit -s 102400
3 g++ -std=c++17 -O2 -Wall "$1" -o main
4 ./main < in.txt > out.txt

```

```

1 # Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
2 # new file : `type nul > filename.cpp`
3 # run : `.\run.ps1 filename.cpp`
4 g++ -std=c++17 -O2 -Wall $args[0] -o main
5 cat in.txt | .\main | Out-File -FilePath out.txt

```

6.4 Comparison

```

1 #!/bin/bash
2 for ((i=1; ; i++)); do
3     ./gen > input.txt # generator
4     ./code < input.txt > output1.txt # your code
5     ./std < input.txt > output2.txt # std code
6     if diff output1.txt output2.txt; then # compare
7         echo "Test $i: Accepted"
8     else
9         echo "Test $i: Wrong Answer"
10        echo "Input:"
11        cat input.txt
12        echo "Your Output:"
13        cat output1.txt
14        echo "Standard Output:"
15        cat output2.txt
16        break
17    fi
18 done

```

6.5 Python

```

1 import sys
2 sys.set_int_max_str_digits(value) # set big number
3 sys.setrecursionlimit(deep) # set deeper recursion
4 lst = sorted(lst, key = lambda x : x[1]) # sort by the second element

```